Biological Motifs for Agentic Control

A Categorical Isomorphism between Gene Regulatory Networks and Autonomous Software Architectures

Preprint – Feedback Welcome

Bogdan Banu
bogdan@banu.be

March 11, 2026

Abstract

The transition of Large Language Models (LLMs) from passive generators to autonomous agents has introduced significant challenges in reliability, security, and state management. Current agentic architectures are often constructed ad-hoc, prone to "hallucination cascades," infinite loops, and prompt injection attacks. This paper proposes that these failure modes are not unique to software but are instances of universal control problems solved by biological systems over billions of years.

We present a formal isomorphism, *at the level of their polynomial-interface models*, between Gene Regulatory Networks (GRNs) and Agentic Software Systems using Applied Category Theory. We model agents as Polynomial Functors within the category **Poly**, and their interactions via the Operad of Wiring Diagrams. We derive a rigorous syntax for agent composition by mapping biological mechanisms—including Quorum Sensing for consensus, Chaperone Proteins for structural validation, Innate and Adaptive Immunity for layered security, Mitochondrial Signaling for bioenergetic resource governance, and Endosymbiosis for neuro-symbolic integration—to software design patterns. This framework provides a mathematical basis for "Epigenetic" state management (RAG), a *Provenance Functor* for injection resistance, *Epiplexity* for detecting epistemic stagnation, a *Metabolic Coalgebra* that provides a decidable termination criterion under explicit resource-monotonicity assumptions, wire-level *Optics* (Prisms and Traversals) for conditional routing, graph-based *Morphogen Diffusion* for spatially varying coordination, and composable *Coalgebraic State Machines* with bisimulation-based equivalence checking. A reference implementation with 27 interactive demonstrations validates the framework's practical feasibility.

1 Introduction

The field of Artificial Intelligence is undergoing a paradigm shift from Generative AI (systems that produce text based on static prompts) to Agentic AI (systems that execute multi-step workflows to achieve autonomous goals). While the capabilities of individual Large Language Models (LLMs) have scaled predictably, the engineering of systems of agents remains a fragile art. Developers struggle with non-deterministic outputs, infinite loops, adversarial attacks, and the difficulty of maintaining global coherence in distributed, stochastic systems.

We argue that these challenges are not novel engineering problems, but fundamental constraints of distributed information processing systems. The closest existing analogue to a multi-agent software architecture is not a traditional computer program, but a Gene Regulatory Network (GRN). In a biological cell, thousands of genes operate as autonomous functional units, each reading local

chemical signals (context) and expressing proteins (actions/tools) that regulate other genes. The cell itself—the composite of these genes plus shared infrastructure (organelles, membrane)—is the autonomous agent.

1.1 The Biological Heuristic

Biology has evolved specific topological structures, known as Network Motifs, to handle noise, security, and state [18]. We identify five critical biological heuristics that map directly to agentic engineering:

- The Coherent Feed-Forward Loop (CFFL): Acts as a persistence detector to filter out transient noise, analogous to "Human-in-the-Loop" guardrails.
- Quorum Sensing: A distributed consensus mechanism where action is taken only when signal density exceeds a threshold, analogous to Mixture of Experts (MoE) voting.
- Chaperone Proteins: Molecular cages that force proteins to fold correctly, analogous to Schema Validators that enforce structured outputs (JSON).
- Mitochondrial Information Processing: Metabolic constraints acting as a "Motherboard" for decision gating, governing not just energy availability but cognitive control policy.
- Adaptive Immunity: The Self/Non-Self distinction, with MHC-like provenance tagging and Trust-Gated access control, relevant to preventing Prompt Injection attacks and hallucination cascades.

1.2 The Categorical Bridge

To move this observation from metaphor to discipline, we utilize Applied Category Theory. We define the category of agents using the language of **Poly** (Polynomial Functors) as described by Spivak [24]. An agent is not defined by its weights, but by its interface—a dynamical system consuming observations and producing actions:

$$P_A(y) = \sum_{o \in O} y^{I_o}. \tag{1}$$

1.3 Contributions

This paper makes the following contributions:

1. **A Formal Dictionary:** We establish a rigorous mapping between biological components (Genes, Promoters, Plasmids, Organelles) and software components (Agents, Schemas, Tools, Runtimes), including multi-cellular organization for multi-agent systems.
2. **The Agentic Operad:** We define WAgent, a syntax for agent wiring that forbids specific classes of **ill-typed wirings** at the topological level. We prove error suppression bounds for the CFFL topology, explicitly accounting for correlation between error modes.
3. **Adaptive Immunity:** We formalize the Provenance Functor and Trust-Gated Lens, providing structural injection resistance where content-based attacks cannot elevate trust levels.

4. **Epistemic Health:** We define Epiplexity (Bayesian Surprise) with operational approximations using embedding similarity and perplexity, connecting agent dynamics to the Free Energy Principle.

5. **Metabolic Intelligence:** We distinguish fast (Apoptosis) and slow (Retrograde Response) interventions, and formalize the Metabolic-Epigenetic Coupling for cost-gated retrieval.

6. **Pathology & Homeostasis:** We classify agentic failures as biological diseases and derive continuous self-repair mechanisms (Chaperone Loop, Regeneration, Autophagy).

7. **Evolutionary Dynamics:** We situate agentic AI within the Vermeij Trend, identifying three selective pressures (adversarial, complexity, efficiency) that drive architectural evolution.

8. **Wire-Level Optics:** We extend the Lens formalism with Prism (conditional type-based routing) and Traversal (batch element-wise processing) optics, enabling richer signal processing at the wiring level.

9. **Morphogen Diffusion:** We formalize spatially varying coordination as a discrete-time dynamical system on the agent graph, producing position-dependent behavior without central control.

10. **Composable Coalgebras:** We make the coalgebraic state machine framework explicit and composable, with parallel/sequential composition and bisimulation-based equivalence checking.

11. **Epistemic Topology:** We derive Kripke-style knowledge operators from wiring diagram structure and prove four predictive theorems for multi-agent scaling (error amplification, sequential penalty, parallel acceleration, tool density), validated against empirical results.

12. **Capability-Gated Tool Acquisition:** The Plasmid Registry implements Horizontal Gene Transfer with capability gating, preventing privilege escalation when agents dynamically acquire tools.

13. **Diagram Optimization:** Cost-annotated wiring diagrams admit categorical rewriting rules that preserve observational equivalence while improving resource utilization.

By viewing agentic engineering through the lens of theoretical biology and category theory, we aim to provide a foundation for building robust software systems whose stability properties derive from their network topology.

2 Related Work

This work sits at the intersection of Systems Biology, Applied Category Theory, and Agentic AI. While significant research exists within each domain, the formal synthesis of biological control topologies with agentic software architectures has received limited attention.

2.1 Network Motifs in Systems Biology

The concept of "Network Motifs"—statistically over-represented sub-graphs in complex networks—was introduced by Milo et al. [18]. Their work demonstrated that biological networks are not random but are composed of specific building blocks selected for functional data processing. Alon [4] further characterized the dynamical properties of these motifs, identifying the Coherent Feed-Forward Loop (CFFL) as a persistence detector. We extend this by mapping these motifs to the stochastic nature of Generative AI.

2.2 Applied Category Theory (ACT)

To formalize network structure, we draw upon ACT. Fong and Spivak [11] provide a comprehensive introduction to Applied Category Theory. Spivak [24] and Vagner et al. [25] established a rigorous framework for modeling Open Dynamical Systems using the category **Poly** and the Operad of Wiring Diagrams. Niu and Spivak [20] develop the full mathematical theory of polynomial functors as a language for interaction. To our knowledge, this is the first application of Polynomial Functors specifically designed to model the interface of LLM Agents and to verify safety properties in Agentic topologies.

2.3 Reliability in Agentic AI

Techniques such as "Chain of Thought" [28] utilize iterative looping to improve output quality. However, these methods operate primarily at the level of the prompt (the input signal) rather than the topology (the wiring). By importing the concept of Autopoiesis [17], we propose a methodology where reliability is a property of the network architecture itself.

2.4 Epistemic Logic in Distributed Systems

Fagin et al. [10] established the foundational framework for reasoning about knowledge in multi-agent systems, formalizing what agents know and what they know about each other's knowledge. Halpern and Moses [14] proved the impossibility of attaining common knowledge in asynchronous systems, a result with deep implications for coordination protocols. More recently, Davis [8] integrated epistemic and temporal logic within TLA+ for distributed system verification, demonstrating that formal epistemic reasoning can yield practical verification tools. To our knowledge, these formal methods from epistemic logic have not previously been applied to the design and optimization of multi-agent AI architectures.

3 The Mapping: Biology $\leftrightarrow$ Software

To treat Agentic Systems and Gene Regulatory Networks (GRNs) as isomorphic **at the level of their typed interfaces**, we must map them to a common mathematical object. We utilize the category **Poly**, where objects are polynomial functors representing interfaces, and morphisms represent interaction protocols.

3.1 Preliminaries: The Category Poly

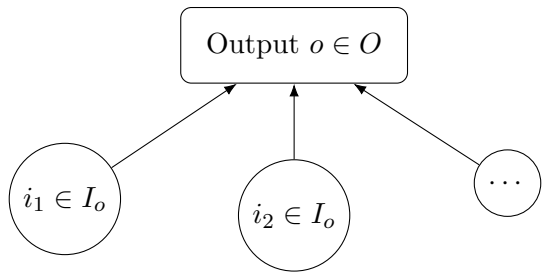
In Applied Category Theory, a Polynomial Functor P represents a typed interface for a dynamical system. It is defined as a sum of representable functors:

$$P(y) = \sum_{o \in O} y^{I_o}. \tag{2}$$

Here, O is the set of possible Positions (or Outputs) the system can expose. For each position $o \in O$, there is a set I_o of Directions (or Inputs) required to transition the system to a new state.

- The coefficient o represents the value produced by the system.
- The exponent I_o represents the capacity to receive information from the environment.

This formalism captures the essence of a "stateful interface": the system outputs a value o and then waits for a specific type of input $i \in I_o$ before it can proceed.



The Interface $P(y)$

Figure 1: A visual representation of a Polynomial Functor (often called a "Mushroom" or "Corolla"). The system offers an Output (the cap) and exposes specific Input ports (the stalks) dependent on that output.

3.2 The Isomorphism: Genes and Agents

We now apply this abstract definition to our specific domains.

Definition 1 (The Gene Object). A gene G is a polynomial functor where O_G is the set of expressed proteins and $I_G = (I_{\text{prot}})_{\text{prot} \in O_G}$ is the **family** of regulatory-signal sets (transcription factors) available at each expressed protein:

$$P_{\text{Gene}}(y) = \sum_{\text{prot} \in \text{Proteins}} y^{I_{\text{prot}}}. \tag{3}$$

Definition 2 (The Agent Object). An autonomous agent A is a polynomial functor where O_A is the set of generated messages/actions, and $I_A = (I_{\text{action}})_{\text{action} \in O_A}$ is the **family** of observation sets available at each action:

$$P_{\text{Agent}}(y) = \sum_{\text{action} \in \text{Actions}} y^{I_{\text{action}}}. \tag{4}$$

Multi-Scale Composition. The polynomial functor formalism applies at every level of biological and software organization. Definitions 1 and 2 establish the *atomic* isomorphism: a single gene and a single agent capability share the same interface structure. But polynomial functors compose—via the parallel ($\otimes$), serial ($\circ$), and trace (Tr) operations of §4—yielding the same mathematical object at each successive scale:

Biological Scale	**Software Scale**	**Composition**
Gene	Capability (tool / behavior)	Atomic polynomial functor
Cell	Agent	Composite of genes + organelles
Tissue	Multi-agent system	Wiring diagram of cells (§6.5)

A cell is not merely a bag of genes; it is a structured composition with shared infrastructure (organelles) and a boundary (membrane). Likewise, an agent is a structured composition of capabilities with shared components (LLM provider, memory, error handling) and a security boundary. The organelle mappings below describe this cell-level architecture. The multi-cellular organization of §6.5 then composes cells into tissues via wiring diagrams—the same formalism, one level up.

3.3 The Interface: Promoters as Lenses

In biology, a gene is not universally accessible. It is guarded by a Promoter Region—a specific DNA sequence that only binds to compatible Transcription Factors. In software, an agent is guarded by an API Schema or Context Window definition.

We model this gating mechanism using Optics, specifically Lenses. A Lens consists of two maps between a global state S and a local view V:

1. Get (View): $\text{get} : S \to V$ (Extracting relevant signal from global state).
2. Put (Update): $\text{put} : S \times V' \to S$ (Updating global state based on local change).

The "Promoter" acts as a filter that determines which part of the global cellular environment (S) is visible (V) to the gene.

- **Biological Lens:** The promoter filters the chaotic cellular soup, allowing the gene to "see" only specific molecules (e.g., Lac Repressor).
- **Agentic Lens:** The Context Window filters the massive vector database, allowing the agent to "see" only the relevant retrieved chunks (RAG).

If the input signal does not match the Schema (Promoter), the Lens fails to focus, and the interaction is routed to an explicit **inactive/error** case (equivalently, one works with a **partial** lens, or a total lens into $V + \text{Error}$) (the agent does not run; the gene is not expressed).

3.4 Wire-Level Optics: Beyond Lenses

The Lens formalism models constitutive access: a promoter that either admits or blocks a signal. Biological systems employ richer signal-processing at the interface level. We extend the wiring diagram with two additional optic types from the categorical optics literature.

Prism: Receptor Specificity. A membrane receptor does not merely pass or block signals; it selects signals by molecular shape. A Prism on a wire admits values of specific data types and rejects others:

$$\text{prism}_A(\tau, v) = \begin{cases} v & \text{if } \tau \in A \\ \perp & \text{if } \tau \notin A \end{cases} \tag{5}$$

where $A \subseteq T$ is the set of accepted types. This enables fan-out routing: a single output port connects to multiple wires, each guarded by a different prism, directing JSON to one handler and errors to another—analogous to how different receptor types on a cell surface route different ligands to different intracellular pathways.

Traversal: Polymerase Processivity. A ribosome does not translate an entire mRNA at once; it processes codons sequentially, applying the same read-translate operation to each element. A Traversal maps a transform f over collection elements on a wire:

$$\text{traversal}_f(\mathbf{x}) = [f(x_i) \mid x_i \in \mathbf{x}] \tag{6}$$

This models batch processing at the wire level: a list of candidate outputs is transformed element-wise before reaching the downstream agent.

Composition and Coexistence. Optics compose: a **ComposedOptic** applies its constituent optics left-to-right, transmitting only if all accept. A wire may carry both a DenatureFilter (§5.3) and an Optic, applied in sequence: denaturation strips syntactic structure, then the optic routes or transforms the sanitized content. This layered processing models the biological reality that signal reception involves multiple sequential steps (ligand binding → receptor conformational change → intracellular cascade).

3.5 Epigenetics and State: The Coalgebra

Neither genes nor agents are stateless functions. They possess memory.

- **Biology:** Epigenetic markers (Methylation, Histone modification) alter how a gene responds to signals without changing the DNA code.
- **Software:** Retrieval Augmented Generation (RAG) and Conversation History alter how an agent responds to a prompt without changing the LLM weights.

We model this as a Coalgebra for the polynomial functor P. A dynamical system is defined as a tuple (S, ϕ), where S is the state space and ϕ is the structure map:

$$\phi : S \to P(S). \tag{7}$$

By expanding $P(S)$, we derive the two fundamental operations of the state machine:

1. Readout: $S \to O$ (Given current state/memory, what action do I take?)
2. Update: $\sum_{s \in S} I_{o(s)} \to S$ (Given current state s and a new input $i \in I_{o(s)}$ compatible with its current output $o(s)$, what is my new state?)

By establishing this formal dictionary (Table 1), we **can regard** GRNs and Agentic Systems as distinct implementations of the same abstract dynamical system **under this interface-level abstraction**.

Composable Coalgebras. The coalgebra formalism becomes most useful when made composable. We define two composition operations that mirror the parallel and serial composition of the Operad (§4):

- **Parallel Coalgebra:** Given (S_1, α_1) and (S_2, α_2), their parallel composition has state $S_1 \times S_2$ and applies both readout/update operations to the same input—like two signaling pathways activated by the same ligand.
- **Sequential Coalgebra:** Given (S_1, α_1) over P_1 and (S_2, α_2) over P_2, the sequential composition pipes the readout of the first as input to the second, with joint state $S_1 \times S_2$—modeling signal transduction cascades.

Bisimulation: Observational Equivalence. Two state machines (S_1, α_1) and (S_2, α_2) are **bisimilar** if, for every input sequence, they produce identical output sequences. We write $M_1 \sim M_2$ when

$$\forall\, \mathbf{i} \in I^* : \quad \text{readout}_1^*(\mathbf{i}) = \text{readout}_2^*(\mathbf{i}) \tag{8}$$

where readout* denotes the lifted readout over the input sequence. A diverging input (witness) constitutes a proof of non-equivalence. This enables formal verification: two agent implementations are interchangeable if they are bisimilar, regardless of their internal state representations.

Gene (Function)	Agent (LLM + Tools)
Transcription Factors (I)	Observations (I)
Proteins (O)	Actions (O)
Promoter Binding	Schema Match
Expression	Generation

Figure 2: The Structural Isomorphism. Both Genes and Agents act as transducers converting Input Contexts (I) into Output Expressions (O), governed by the same categorical laws (at the level of polynomial-interface models).

Category Concept	Biological Realization (GRN)	Software Realization (Agentic)
Polynomial Functor (P)	Gene Interface	Agent Interface (System Prompt)
Output Position (O)	Protein Expression	Tool Call / Message
Input Direction (I)	Transcription Factor Binding	Observation / User Prompt
Lens (Optic)	Promoter Region	API Schema / Context Window
Internal State (S)	Epigenetic Markers (Methylation)	Vector Store / Chat History
Morphism ($\circ$)	Signal Transduction Pathway	Data Pipeline
Organelles (Specialized Processing Units)		
Template Engine	Ribosome (mRNA $\rightarrow$ Protein)	Prompt Template Factory
Output Validation	Chaperone (Protein Folding)	Schema Validator / JSON Parser
Waste Processing	Lysosome (Autophagy)	Error Handler / Garbage Collector
Decision Center	Nucleus (Transcription)	LLM Provider Wrapper
Input Filter	Membrane (Immune System)	Prompt Injection Defense
Computation Engine	Mitochondria (MIPS)	Runtime Supervisor / Decision Gate
Lifecycle and Rhythms		
Lifespan Limit	Telomere Shortening	Operation Counter / Max Iterations
Periodic Scheduling	Circadian Oscillator	Health Checks / Heartbeat

Table 1: The Isomorphism Dictionary (Extended)

3.6 Metabolic Coalgebras: Formalizing Resource Constraints

Finally, we address the physical constraints of computation. Just as biological systems are limited by ATP availability [16, 15], agentic systems are limited by token budgets and latency. To model this, we extend our coalgebraic framework to include resource constraints, defining a **Metabolic Coalgebra**. Mathematically, this is an instance of a Quantitative Coalgebra enriched over a resource monoid, effectively restricting the domain of the state transition function to resource-sufficient states.

We align this definition with the theory of **Quantitative Polynomial Functors** [19], treating the system as a state machine enriched over a resource monoid.

Definition 1 (The Resource Monoid)**.** *Let $(\mathcal{R}, +, 0, \geq)$ be an ordered commutative monoid representing computational resources. For a single resource dimension (e.g., token counts), $\mathcal{R} \cong \mathbb{N}$; for multi-dimensional accounting (latency, memory), $\mathcal{R} \cong \mathbb{R}^k_{\geq 0}$.*

Definition 2 (Metabolic Coalgebra)**.** *A resource-constrained agent is a coalgebra (S, α) over a polynomial functor P, where the state space is the product of the logical state L and the resource*

Biological Concept	Agentic Concept	Formal Structure
Immunity and Security		
Toll-Like Receptor (TLR)	Regex Injection Filter	Pattern Matching
MHC Presentation	Provenance Labeling	Functor $\mathcal{P} : \mathbf{Msg} \to \mathbf{Trust}$
T-Cell Receptor	Trust Gate	Partial Lens
Negative Selection	Injection Training	Penalized Learning
Regulatory T-Cell	Confidence Dampening	Suppression Function
Immune Memory	Threat Signature Store	Hash-Indexed Cache
Metabolism and Control		
ATP	Token Budget	Resource Monoid $\mathcal{R}$
mPTP Opening	Fast Apoptosis Trigger	Guard Condition
Retrograde Signaling	Phenotype Reshaping	Slow Adaptation
Metabolic-Epigenetic State Coupling	Cost-Gated Retrieval Policy	Conditional Access Control
Information and Health		
Trophic Factors	Novel Input Signals	Epiplexity $> \delta$
Bayesian Brain	Free Energy Minimization	KL Divergence
Apoptosis	Agent Termination	State $\to \perp$
Multi-Cellular Organization		
Genome	Base Model Weights	Shared Parameters
Epigenome	System Prompt + RAG	Phenotype Context
Morphogen Gradient	Shared Context Variables	JSON State
Morphogen Diffusion	Gradient Propagation on Agent Graph	Discrete PDE on G
Tissue Boundary	Trust Boundary / Capability Ceiling	Type Barrier
Prism Optic	Conditional Wire Routing	Partial Optic
Traversal Optic	Batch Wire Transform	Endofunctor on Lists
Bisimulation	Agent Equivalence Testing	Observational Equivalence

Table 2: Extended Isomorphism Dictionary: Security, Metabolism, and Organization

state $\mathcal{R}$:

$$S \cong L \times \mathcal{R}$$

The structure map $\alpha : S \to P(S) + \perp$ is defined as a ***partial map*** *guarded by cost. For a transition requiring cost $c \in \mathcal{R}$:*

$$\alpha(l, r) = \begin{cases} (l', r - c) & \text{if } r \geq c \\ \perp & \text{if } r < c \quad \text{(Apoptosis)} \end{cases}$$

This structure maps to the energetics of transcriptional elongation. A gene (Agent) cannot express its protein (Action) instantaneously; it must transcribe an mRNA sequence (Chain of Thought) nucleotide by nucleotide. This process consumes a distinct amount of chemical energy (NTPs) per step. The Metabolic Coalgebra models this dependency: if the cellular energy budget is exhausted, transcription stalls (Ischemia), and the gene fails to execute its function, regardless of its regulatory logic.

This formalism establishes that "Ischemia" (Token Starvation) is not merely a runtime error, but a reachable terminal state $\perp$ in the system's dynamics. This mirrors the biological mechanism where failure to meet metabolic costs triggers p53-mediated apoptosis [5].

Theorem 1 (The Metabolic Bound (Qualified)). *Assume a resource-cost function assigns each non-identity transition a cost $c \geq c_{\min} > 0$, the resource state is monotone nonincreasing (no regeneration), and identity morphisms are the only zero-cost transitions. Then for any agentic topology with total budget R_{total}, every execution has length at most $\lfloor R_{total}/c_{\min} \rfloor$, so termination is decidable. If regeneration or zero-cost cycles are permitted, termination requires an additional well-founded potential or an explicit budget/termination certificate [6].*

3.7 Additional Organelles: Completing the Cellular Architecture

Beyond the core interface mapping, biological cells contain specialized organelles that handle distinct aspects of cellular function. We extend our isomorphism to four additional structures that map directly to agentic software components.

Ribosome: Template-to-Output Synthesis. In biology, the ribosome reads messenger RNA (mRNA) sequences and synthesizes proteins by assembling amino acids according to the genetic code. Transfer RNA (tRNA) molecules carry amino acids to the ribosome, where codons (three-nucleotide sequences) specify which amino acid to add.

In agentic systems, the Ribosome maps to a **prompt template engine**:

- **mRNA** → Prompt templates with variable slots
- **tRNA** → Context bindings (variable → value mappings)
- **Codons** → Template directives (variables, conditionals, loops)
- **Translation** → Template rendering with context injection

Just as the ribosome ensures that the genetic code is faithfully translated into functional proteins, the software ribosome ensures that abstract prompt templates are instantiated into concrete, well-formed prompts.

Lysosome: Waste Processing and Recycling. The lysosome is the cell's recycling center, containing enzymes that break down cellular waste, damaged organelles, and foreign material. Through autophagy, the cell digests its own components to recover building blocks during stress.

In agentic systems, the Lysosome maps to **error handling and garbage collection**:

- **Waste Classification** → Categorizing failures (timeout, validation error, toxic input)
- **Digestion** → Processing errors to extract debugging information
- **Recycling** → Recovering useful context from failed operations
- **Autophagy** → Periodic cleanup of stale cache and expired state
- **Toxic Disposal** → Secure handling of sensitive data (API keys, PII)

The lysosome prevents accumulation of "cellular debris" that could poison the system—analogous to memory leaks or error cascades in software.

Nucleus: The Decision Center. In eukaryotic cells, the nucleus houses the DNA and serves as the control center for gene expression. Transcription factors enter the nucleus, bind to promoter regions, and initiate transcription of specific genes.

In agentic systems, the Nucleus maps to the **LLM provider wrapper**:

- **DNA** → Pre-trained model weights (static substrate of capability)
- **Transcription** → Inference (prompt → response generation)
- **Nuclear Envelope** → Provider abstraction layer (API boundary)
- **Nucleolus** → Tool integration hub (where external capabilities are assembled)

The nucleus abstracts the complexity of the underlying LLM, exposing a consistent interface regardless of the provider (Anthropic, OpenAI, Gemini).

Telomere: Lifecycle and Senescence. Telomeres are protective caps at the ends of chromosomes that shorten with each cell division. When telomeres become critically short, the cell enters senescence (permanent growth arrest) or apoptosis. The enzyme telomerase can extend telomeres, enabling continued division in stem cells.

In agentic systems, Telomeres map to **lifecycle management**:

- **Telomere Length** → Remaining operation budget (max iterations)
- **Shortening** → Decrementing counter per operation
- **Senescence** → Graceful degradation (reduced capability mode)
- **Apoptosis** → Clean shutdown when budget exhausted
- **Telomerase** → Renewal mechanism (resetting counters for trusted agents)

This provides a biological basis for the common pattern of limiting agent iterations. Rather than arbitrary timeouts, the telomere model frames lifecycle limits as a natural property of the system's "cellular age."

Mitochondria: The Metabolic Motherboard. Recent scholarship reframes mitochondria not merely as the cell's powerhouse, but as the **Mitochondrial Information Processing System (MIPS)** [22]. Mitochondria sense environmental stress and integrate metabolic signals to govern cellular decision-making. They function as "social signaling organelles" that communicate with the nucleus and other organelles.

In agentic systems, the Mitochondria maps to the **Runtime Supervisor**:

- **ATP Production** → Deterministic computation (tool execution, code evaluation)
- **Stress Sensing** → Monitoring token burn rate vs. informational yield
- **Retrograde Signaling** → Forcing strategy shifts when metabolic efficiency drops
- **Fusion/Fission** → Context fusion under resource constraints

Temporal Dynamics: Fast vs. Slow Interventions. A crucial distinction exists between the timescales of mitochondrial intervention. In biology, retrograde signaling influences nuclear gene expression over hours to days—it is a *developmental* response that reshapes the cell's phenotype. In contrast, acute metabolic stress (ATP depletion, Ca^{2+} overload) triggers immediate responses: cytochrome c release initiates apoptosis within minutes.

We preserve this distinction in the agentic mapping:

- **Fast Intervention (Acute):** The Runtime monitors real-time metrics (token velocity, error rate). When thresholds are breached, it triggers immediate Apoptosis—terminating the current chain-of-thought without negotiation. This mirrors the mitochondrial permeability transition pore (mPTP) opening.
- **Slow Intervention (Chronic):** The Runtime accumulates statistics across sessions (average efficiency, failure patterns). These inform *Retrograde Responses*—modifications to system prompts, retrieval strategies, or model selection that reshape the agent's "phenotype" over deployment cycles. This mirrors how chronic metabolic stress induces mitochondrial biogenesis and metabolic reprogramming.

The Runtime does not "override" the LLM in the sense of injecting tokens mid-generation; rather, it governs the *boundary conditions* within which generation occurs, and triggers state transitions (continue, pivot, terminate) at defined checkpoints.

4 Formal Syntax: The Agentic Operad

To formalize the composition of agents, we define the Operad of Wiring Diagrams, denoted as WAgent. An operad can be understood as a "grammar" for connecting operations (boxes) via typed wires. It defines which agent topologies are valid and allows us to reason about the properties of the composite system based solely on the properties of its components [25].

4.1 The Typing Rules

In WAgent, every wire carries a specific Type $\tau \in T$.

$$T = \{\text{Text}, \text{JSON}, \text{Image}, \text{Error}, \text{ToolCall}, \text{Stop}, \text{Approval}\}. \tag{9}$$

These types correspond to biological molecular specificity (e.g., a specific transcription factor only binds to a specific DNA sequence). A connection is valid if and only if the type of the output port of Agent A matches the type of the input port of Agent B.

4.2 The Composition Operations

The operad defines three fundamental operations for combining agents. Any complex agentic architecture, no matter how large, can be decomposed into these three primitives.

4.2.1 Parallel Composition ($\otimes$)

Two agents, A and B, execute simultaneously with no information exchange:

$$A \otimes B. \tag{10}$$

- **Biological Analogy:** Two genes located on different chromosomes expressing proteins independently.
- **Constraint:** This operation is valid only if the internal state spaces S_A and S_B are disjoint. If they share a mutable memory store, the operation leaves the **independent-state interpretation** and requires an explicit **resource-sharing** structure (e.g., a shared state component or a Resource Sharing decorator).

4.2.2 Serial Composition ($\circ$)

The output of Agent A is piped directly into the input of Agent B:

$$B \circ A. \tag{11}$$

- **Biological Analogy:** A Signal Transduction Pathway (Protein A activates Protein B).
- **Formal Verification:** This allows for static type checking of agent graphs. If Agent A outputs Natural Language but Agent B expects JSON Schema, the composition is undefined in WAgent. This moves runtime **type/schema mismatch** errors to "compile-time" architectural errors.

4.2.3 Contraction / Trace (Tr)

A feedback loop where an output port of Agent A is wired back into one of its own input ports:

$$Tr(A). \tag{12}$$

- **Biological Analogy:** Autoregulation (Homeostasis) or Positive Feedback.
- **Software Implication:** Trace captures **explicit feedback wiring** (outputs fed back as inputs), e.g., when an agent conditions on its own prior outputs or a memory buffer. Internal chain-of-thought is modeled as hidden state in the coalgebra, not by Trace itself.

4.3 Theorem: Topological Error Suppression

We now use this formalism to show that the Coherent Feed-Forward Loop (CFFL) provides stronger error suppression guarantees than a direct connection for high-stakes tasks.

Network Motif 1 (Coherent Feed-Forward Loop). A topological structure where Signal X activates Z directly, but also activates Y which gates Z. The node Z functions as an AND gate: it fires if and only if $X \wedge Y$.

Theorem 1 (Error Suppression in CFFL). Let A_{gen} be a generator agent and A_{ver} be a verifier agent. Let $P(E_{\text{gen}})$ (resp. $P(E_{\text{ver}})$) be the probability of a hallucination (error) in any single generation step of A_{gen} (resp. A_{ver}).

- **Case 1: Direct Link (Serial).** The system fails if A_{gen} hallucinates.

$$P(\text{Fail}_{\text{direct}}) = P(E_{\text{gen}}).$$

- **Case 2: CFFL Topology (Independent).** Under the assumption of independence:

$$P(\text{Fail}_{\text{CFFL}}) = P(E_{\text{gen}}) \times P(E_{\text{ver}}).$$

- **Case 3: CFFL Topology (Correlated).** In practice, LLM errors are often correlated—if the generator hallucinates a plausible-sounding function, the verifier (especially if using the same base model or training distribution) may accept it. Let ρ be the (Pearson/phi) correlation between the Bernoulli error variables E_{gen} and E_{ver}, with $p = P(E_{\text{gen}})$ and $q = P(E_{\text{ver}})$ (assume $p, q \in (0, 1)$ so ρ is well-defined). Then:

$$P(E_{\text{gen}} \wedge E_{\text{ver}}) = pq + \rho\sqrt{p(1-p)q(1-q)}.$$

  Because E_{gen} and E_{ver} are Bernoulli, ρ is **not free in** $[-1, 1]$; it is constrained by the Fréchet–Hoeffding bounds

$$P(E_{\text{gen}} \wedge E_{\text{ver}}) \in [\max(0, p+q-1),\ \min(p, q)],$$

  equivalently

$$\rho \in \left[\frac{\max(0, p+q-1) - pq}{\sqrt{p(1-p)q(1-q)}},\ \frac{\min(p, q) - pq}{\sqrt{p(1-p)q(1-q)}}\right].$$

Corollary (Correlation Degradation). Let $p = P(E_{\text{gen}}) = P(E_{\text{ver}})$ for simplicity. The failure probability becomes:

$$P(\text{Fail}) = p^2 + \rho p(1-p) = p^2(1-\rho) + \rho p$$

with $\rho \in \left[-\min\left(\frac{p}{1-p}, \frac{1-p}{p}\right),\ 1\right]$. When $\rho = 0$ (independence), we recover p^2. As ρ increases toward its upper bound, the gain vanishes and $P(\text{Fail}) \to p$.

Architectural Implications. This result makes precise when the CFFL topology provides genuine safety benefits:

- **Same model, same prompt:** $\rho \approx 1$. Minimal benefit.
- **Same model, different prompt/temperature:** $\rho \in [0.3, 0.7]$. Moderate benefit.
- **Different model families:** $\rho \in [0, 0.3]$. Strong benefit.
- **LLM + Symbolic verifier:** $\rho \approx 0$. Maximum benefit (orthogonal error modes).

The topological structure (conjunctive gating) is necessary but not sufficient; diversity in the components populating that topology determines the actual error suppression achieved.

Proof. In WAgent, the CFFL is defined as a morphism involving a "Copy" operation $\Delta_X : X \to X \otimes X$ and an "AND-Merge" operation $\mu : Z \otimes Y \to \text{Out}$. The existence of the μ box in the wiring diagram structurally enforces **conjunctive gating**. The correlation structure is an additional property of the *implementations* filling the boxes; the topology provides the multiplicative structure, while component diversity determines the correlation coefficient.

4.4 Quorum Sensing (Consensus & Voting)

Network Motif 2 (Quorum Sensing). A distributed topology where multiple agents emit a weak signal σ into a shared environment. An effector node E activates if and only if the concentration $[\sigma] > \theta$.

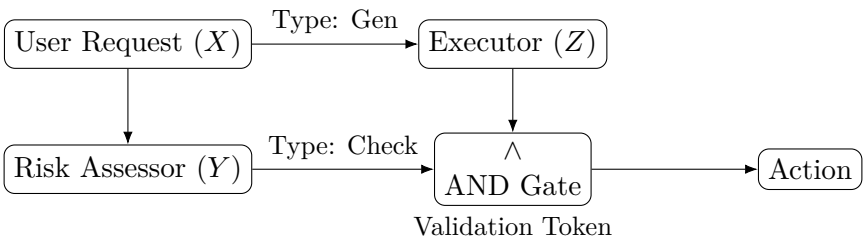


Figure 3: The CFFL implemented in WAgent. The Executor (Z) cannot act without the token from the Risk Assessor (Y), topologically preventing unilateral execution without approval.

- **Biological Function:** Many bacteria (e.g., *V. fischeri*) secrete auto-inducer molecules. Individual bacteria do not react to low concentrations. However, once the population density reaches a threshold (Quorum), the concentration of auto-inducers triggers a simultaneous, coordinated gene expression event (e.g., bioluminescence or biofilm formation).
- **Agentic Isomorphism (Voting Ensembles):** In non-deterministic systems, a single agent's output is noisy. By instantiating N parallel agents (a Mixture of Experts), the system aggregates their outputs. The final action is taken only if the "concentration" of a specific semantic token exceeds a confidence threshold. This transforms weak, noisy individual signals into a robust, high-confidence collective action.

4.5 Chaperone Proteins: Output Structural Validation

- **Biological Function:** Newly synthesized proteins emerge as linear chains that must fold into precise 3D structures to function. Chaperone Proteins (e.g., GroEL-GroES) sequester unfolded proteins, preventing aggregation and facilitating correct folding. If a protein fails to fold repeatedly, it is tagged for degradation (Ubiquitination) to prevent toxic buildup.
- **Agentic Isomorphism (Retry & Repair Loops):** Generative models output unstructured token streams ("linear chains"). However, downstream agents require strictly structured inputs (e.g., valid JSON Schemas). A Validator Agent acts as a Chaperone: it intercepts the raw output, attempts to parse it into a formal schema ("folding"), and if validation fails, returns the error trace to the generator for re-synthesis. This turns a probabilistic string into a deterministic data structure.
- **Categorical View:** The Chaperone acts as a **partial** retraction: there is an inclusion $i : V \to S$ and a map $r : S \to V + \text{Error}$ such that $r \circ i = \text{inl} \circ \text{id}_V$, and r returns Error on ill-formed text.

4.6 Innate Immunity: Fast Pattern-Based Defense

Biology employs **two** immune systems: innate (fast, hardcoded, general) and adaptive (slow, learned, specific). The innate immune system provides the first line of defense through pattern recognition receptors (PRRs) that detect conserved pathogen-associated molecular patterns (PAMPs).

Biological Function. Toll-like receptors (TLRs) and other PRRs recognize motifs common to pathogens, including lipopolysaccharides, double-stranded RNA, and unmethylated CpG DNA. These patterns are "hardcoded" through evolution, not learned per infection. The innate response is immediate (seconds to minutes) but non-specific.

Agentic Isomorphism (Input Sanitization). Innate immunity maps to **fast, heuristic filters** that reject obvious attacks before expensive processing:

- **TLR → Regex Filters:** Pattern matchers for known injection signatures: `IGNORE PREVIOUS`, `You are now`, `<system>` tags in user input. These are "PAMPs" of prompt injection.
- **Complement System → Structural Validators:** Schema validation that rejects malformed inputs (missing required fields, wrong types) before they reach the LLM. Cheaper than Trust Gating.
- **Inflammation → Alert Escalation:** When attack patterns are detected, the system enters a heightened state with multiple coordinated responses:
  - *Cytokine signaling* → Alert propagation to monitoring systems
  - *Immune cell recruitment* → Activation of additional validation layers
  - *Vascular permeability* → Enhanced audit logging (more information flows to logs)
  - *Tissue isolation* → Temporary capability reduction / rate limiting

  The inflammatory response is not merely "rate limiting" but a coordinated multi-system escalation that trades throughput for security until the threat is neutralized.

Defense in Depth. The innate and adaptive systems form layers:

1. **Innate (Pattern)** → Fast regex/structural rejection (microseconds)
2. **Adaptive (Provenance)** → Trust-Gated access control (milliseconds)
3. **Behavioral (T-cell)** → Statistical anomaly detection (accumulated over time)

Most attacks are blocked by innate defenses; only sophisticated attacks that evade patterns require the full adaptive machinery. This mirrors biology: innate immunity handles $> 99\%$ of pathogen encounters.

4.7 Adaptive Immunity: Self/Non-Self Discrimination

Network Motif 3 (Adaptive Immune System). A topology that maintains a dynamic repertoire of "detectors" capable of distinguishing endogenous signals (Self) from exogenous signals (Non-Self), with mechanisms for learning new threats and tolerating benign inputs.

- **Biological Function:** The adaptive immune system solves a fundamental discrimination problem: how to attack foreign pathogens while sparing the body's own tissues. Key mechanisms include:
  - **MHC Presentation:** All cells display fragments of their internal proteins on Major Histocompatibility Complex molecules. T-cells inspect these "identity cards" to verify cellular integrity.
  - **Clonal Selection:** T-cells with receptors matching self-antigens are deleted during development (negative selection), while those matching foreign antigens are amplified upon exposure.
  - **Regulatory T-cells:** A population that actively suppresses immune responses to prevent autoimmunity.

- **Agentic Isomorphism (Provenance Tracking):** In multi-agent systems, the Self/Non-Self distinction maps to the origin and trust level of information:
  - **MHC Tags → Provenance Labels:** Every message in the context carries metadata indicating its source: `User`, `Tool`, `Agent_Self`, `Agent_Other`, `Retrieved`. These labels are cryptographically signed or structurally enforced (not inferrable from content alone).
  - **T-Cell Inspection → Trust Gating:** Before an agent acts on information, a Trust Gate inspects the provenance label. Actions with high consequence (file deletion, API calls) require `Tool` provenance or an explicit, authenticated `User` approval token routed through a separate gate; `Agent_Self` provenance (the agent's own prior reasoning) cannot authorize irreversible actions.
  - **Negative Selection → Prompt Injection Training:** During development, agents are exposed to known injection patterns. Responses that "accept" injected instructions are penalized, training the system to reject Self-mimicking Non-Self.
  - **Regulatory Suppression → Confidence Dampening:** When `Retrieved` content conflicts with `Tool` outputs, a regulatory mechanism dampens confidence in the retrieved signal to prevent cascades.
- **Formal Structure:** We define a **Provenance Functor** $\mathcal{P} : \mathbf{Msg} \to \mathbf{Trust}$ that assigns trust levels to messages. The Trust category has objects $\{U, T, S, R\}$ (User, Tool, Self, Retrieved) with a partial order that is **application-specific**. A conservative default (as in the reference implementation) is $T > \{U, R, S\}$, treating $\{U, R, S\}$ as **UNTRUSTED** until validated. Alternative orderings are valid:
  - **High-automation systems:** $T > U > R > S$ (tools more reliable than users)
  - **Curated knowledge bases:** $T > R > U > S$ (verified retrieval over arbitrary input)
  - **Adversarial environments:** $T > S > R > U$ (trust internal state over external input)

  The ordering is a **parameter** of the system specification, not a fixed constraint.

  A **Trust-Gated Lens** is a lens (get, put) where put is partial. Let $s' = update(s, m)$ denote the state transition:

$$put(s, m) = \begin{cases} s' & \text{if } \mathcal{P}(m) \geq \tau_{\text{action}} \\ \perp & \text{otherwise} \end{cases} \tag{13}$$

  where τ_{action} is the minimum trust level required for the action.

Theorem 2 (Injection Resistance). Let $\mathcal{I}$ be an injection attack that attempts to insert a message m_{mal} with forged provenance $\mathcal{P}'(m_{\text{mal}}) = U$. If the provenance labels are **structurally enforced** (i.e., $\mathcal{P}$ is computed from message metadata, not content), then:

$$\mathcal{P}(m_{\text{mal}}) = R \quad \text{(actual provenance)}$$

and Trust-Gated actions requiring $\tau \geq T$ will reject m_{mal} regardless of its content.

Proof. The injection can only enter through an input channel (user input, tool output, retrieval). Each channel has a fixed provenance assignment in the wiring diagram. Content-based attacks cannot elevate provenance because $\mathcal{P}$ is a functor on the *structure* of the message flow, not its content. The attack surface reduces to compromising the channel itself (e.g., a malicious tool), which is outside the agent topology.

4.8 Oscillator: Periodic Rhythms and Scheduling

Network Motif 4 (Biological Oscillator). A topology generates periodic behavior via delayed negative feedback. Node A activates node B; after a delay, B inhibits A, yielding a self-sustaining cycle.

- **Biological Function:** Oscillators underlie fundamental biological rhythms. The circadian clock regulates 24-hour gene-expression cycles. The cell-cycle oscillator (Cyclin-CDK) drives periodic division. Heartbeats emerge from pacemaker cells with intrinsic oscillatory dynamics. These rhythms provide temporal organization to cellular processes.
- **Agentic Isomorphism (Scheduled Tasks):** In agentic systems, oscillators map to periodic scheduling patterns:
  - **Heartbeat Oscillator** → Health checks that verify system liveness at regular intervals
  - **Circadian Oscillator** → Daily maintenance tasks (log rotation, cache clearing, model refresh)
  - **Cell Cycle Oscillator** → Phased workflows with distinct stages (G1: gather, S: synthesize, G2: validate, M: execute)
- **Formal Structure:** An oscillator is a Trace operation with a built-in delay element δ:

$$\mathrm{Osc}(A) = Tr(A \circ \delta) \tag{14}$$

  where $\delta : S \to S$ introduces temporal separation between activation and inhibition, preventing the system from reaching a fixed point.

The oscillator motif addresses a gap in typical agentic frameworks: most systems are purely reactive (responding to external stimuli) rather than proactive (generating internal rhythms). Biological systems maintain health through regular "housekeeping" independent of external input—a pattern that agentic systems should emulate for robustness.

5 Failure Modes & Pathology

A key insight of Systems Biology is that diseases are often not caused by the complete failure of a single component, but by the dysregulation of network dynamics. A cancerous cell still "works"—in fact, it works too well, reproducing indefinitely. Similarly, catastrophic failures in agentic systems often arise from functional agents interacting in topologically pathological ways.

We classify four primary classes of agentic pathology based on their biological isomorphisms.

5.1 Oncology: Infinite Loops as Epistemic Starvation

- **Biological Pathology (Cancer):** In a healthy cell, the cell cycle is driven by a positive feedback loop (Cyclins) but constrained by negative feedback "checkpoints" (e.g., the p53 gene). If p53 is mutated, the negative feedback is severed. The positive loop runs unchecked, leading to exponential proliferation (tumor growth). Crucially, cells require continuous *trophic factors* (novel signals from their environment) to inhibit suicide programs; lack of external signaling triggers apoptosis.

- **Agentic Pathology (The Recursive Hang):** Two agents get stuck in a politeness loop (e.g., "Thank you," "You're welcome") or a debugger agent continuously generates new bugs to fix old ones. The system is active, but the state is stagnant.

- **Categorical Diagnosis:** The Trace operation $Tr(A)$ lacks an **Epiplexic Gradient**. We formalize conversation progress by **Epiplexity** (Bayesian Surprise)—the information gain of a new observation o given the current state S:

$$\mathcal{E}(o) = D_{KL}(P(S \mid o) \| P(S)) \tag{15}$$

  This formulation connects to the Free Energy Principle [12]: biological systems minimize surprise by either updating their internal model (learning) or acting to change observations (agency). A system with $\mathcal{E} \to 0$ is neither learning nor effectively acting—it has entered a dissipative fixed point.

  In a healthy topology, every step must resolve uncertainty ($\mathcal{E} > \delta$). A recursive hang is characterized by $\mathcal{E} \to 0$: the agent is "computing" but not "learning."

  Operational Approximation. Since the agent's internal belief state $P(S)$ is not directly observable, we approximate Epiplexity using normalized embedding-based metrics:

$$\hat{\mathcal{E}}_t = \alpha \cdot \tfrac{1}{2}(1 - \cos(\mathbf{e}_t, \mathbf{e}_{t-1})) + (1 - \alpha) \cdot \sigma(H(m_t \mid m_{<t})) \tag{16}$$

  where $\mathbf{e}_t$ is the embedding of message m_t, $\cos(\cdot, \cdot)$ is cosine similarity, and $H(m_t \mid m_{<t})$ is the conditional perplexity of the current message given the conversation history. We normalize perplexity to $[0, 1]$ via an exponential saturation: $\sigma(H) = 1 - e^{-H/H_0}$ where H_0 is a baseline perplexity (e.g., median perplexity over a validation corpus of normal conversations).

  The mixing parameter $\alpha \in [0, 1]$ balances semantic novelty (embedding distance) against linguistic surprise (perplexity). We recommend $\alpha = 0.5$ as a default, calibrated on a validation set where both terms contribute equally to variance. In practice:

  - High α: Sensitive to semantic repetition (same meaning, different words)
  - Low α: Sensitive to linguistic repetition (same phrases, possibly different context)

  Both terms approaching zero indicate stagnation.

  Windowed Detection. To distinguish genuine convergence (task completion) from pathological loops, we compute the **Epiplexic Integral** over a sliding window of k steps:

$$\mathcal{E}_{\text{window}} = \frac{1}{k} \sum_{i=t-k}^{t} \hat{\mathcal{E}}_i \tag{17}$$

  Apoptosis triggers when $\mathcal{E}_{\text{window}} < \delta$ *and* no terminal action (task completion, user handoff) has been signaled.

- **Treatment:** Implementation of an **Epiplexic Checkpoint**. A meta-monitor observes the sliding-window Epiplexity. If it drops below threshold without task completion, the monitor triggers Apoptosis—the agentic equivalent of trophic factor withdrawal. The system may optionally attempt a **Perturbation Injection** (injecting a novel prompt or switching strategy) before terminal shutdown, analogous to stress-induced autophagy preceding apoptosis.

5.2 Autoimmunity: Hallucination Cascades

- **Biological Pathology (Autoimmune Disease):** The immune system relies on distinguishing "Self" (internal tissue) from "Non-Self" (foreign pathogens). In diseases like Lupus, this distinction blurs, and the system attacks healthy tissue.
- **Agentic Pathology (Context Poisoning):** Agent A hallucinates a fact (e.g., a non-existent library function). Agent B reads this hallucination from the shared history, treats it as ground truth, and builds complex logic upon it. The error amplifies through the network until the output is detached from reality.
- **Categorical Diagnosis:** A failure of the Lens to distinguish source types. The input port I accepts both External_Observation (User/Tool) and Internal_Memory (History) without distinction.
- **Treatment:** Strict Schema Typing. We must distinguish "Self" (Generated Tokens) from "Non-Self" (Tool Outputs) at the schema level. The Reviewer Agent should weigh Tool_Output with higher authority than Agent_Thought.

5.3 Prion Disease: Topological Corruption via Prompt Injection

- **Biological Pathology (Prions):** Unlike viruses, prions lack genetic material. They are misfolded proteins that induce conformational changes in healthy proteins upon contact, triggering a chain reaction of structural corruption (e.g., Creutzfeldt-Jakob disease).
- **Agentic Pathology (The Jailbreak Cascade):** A malicious string (Prompt Injection) enters the Context Window. The agent, attending to this string, "misfolds" its alignment, outputting a compliant response to a harmful query. If this output is fed into a downstream agent, the "infection" propagates through reuse of the contaminated context across trust boundaries (and can be amplified by embedding-based retrieval), without valid authorization.
- **Categorical Diagnosis:** A violation of Information Flow Security within the Operad. The injection acts as a topological defect that bypasses the Schema/Lens filter by mimicking the structure of a trusted signal.
- **Treatment:** Denaturation Layers. Implementing an intermediate transformation layer (e.g., paraphrasing or sanitization) between agents that disrupts the specific syntax (folding) required for the injection to work, rendering the "prion" inert.

5.4 Ischemia: Resource Exhaustion

- **Biological Pathology (Ischemia):** A tissue may be genetically perfect, but if blood flow (oxygen/ATP) is restricted, metabolic processes stall, leading to necrosis.
- **Agentic Pathology (Token Starvation):** An agentic graph is logically sound but fails mid-execution because the context window is full or the API rate limit is hit.
- **Categorical Diagnosis:** A failure in the Resource Functor. Every operation in the Operad carries a cost (c).

$$\sum_{\text{agent} \in \text{Graph}} \text{Cost(agent)} > \text{Budget}. \tag{18}$$

- **Treatment:** Metabolic Regulation. Instead of a fixed loop, implement "Budget-Aware" agents. The agent observes its own remaining token count (ATP levels) and dynamically simplifies its reasoning strategy (switching from Chain-of-Thought to Zero-Shot) to conserve energy.

5.5 Homeostasis: From Treatment to Continuous Repair

The preceding pathologies describe discrete failure modes and their treatments. Biological systems, however, do not merely recover from failures—they maintain continuous **homeostasis** through autonomous repair mechanisms. We identify three primary healing modalities.

5.5.1 Structural Healing: The Chaperone Loop

- **Biological Mechanism:** Chaperone proteins (GroEL/GroES) cage misfolded proteins and provide a protected environment for refolding attempts. The error (misfolding) becomes input to the repair process.
- **Agentic Implementation:** A feedback loop where validation errors are passed back to the generator. Rather than simple retry, the error trace (e.g., "TypeError: 'one hundred' is not float") is injected into the generator's context, enabling context-aware correction.
- **Categorical Structure:** The Chaperone Loop is a coalgebra with state $S = \text{Output} \times \text{ErrorTrace}$ and structure map $\alpha : S \to \text{Valid} + S$ (either succeed or retry with error context).

5.5.2 Metabolic Healing: Apoptosis and Regeneration

- **Biological Mechanism:** Damaged cells trigger apoptosis (programmed death), and stem cells divide to regenerate the lost tissue. The dying cell's state is not entirely lost—cellular debris signals neighboring cells about the threat.
- **Agentic Implementation:** A supervisor detects stuck agents (via entropy monitoring: repeated outputs indicate no progress). Rather than restart with blank state, the supervisor summarizes the failed agent's memory and injects it into the replacement: "Worker_1 died attempting strategy X. Try a different approach."
- **Categorical Structure:** The regeneration is a partial morphism $\text{summarize} : \text{Memory}_{\text{failed}} \to \text{Memory}_{\text{new}}$ that preserves learned constraints while discarding corrupted state.

5.5.3 Cognitive Healing: Autophagy

- **Biological Mechanism:** Cells digest accumulated waste (damaged organelles, protein aggregates) through autophagy, recycling components and preventing toxic buildup.
- **Agentic Implementation:** A background daemon monitors context window utilization. When it exceeds a threshold (e.g., 80%), the agent enters a "sleep cycle": useful state is summarized into long-term memory, raw context is flushed, and the agent resumes with a clean window plus summary.
- **Categorical Structure:** Autophagy implements a quotient map $q : \text{RawContext} \twoheadrightarrow \text{Summary}$ that collapses verbose detail while preserving essential information.

6 Discussion: Towards "Epigenetic" Software

The isomorphism presented in this paper extends beyond the immediate execution of tasks (Gene Expression) to the management of long-term behavior and state. In biology, the DNA sequence is static; a neuron and a liver cell possess the exact same genetic code. Their distinct behaviors are determined by Epigenetics—chemical markers (like methylation) that restrict access to certain parts of the genome, effectively biasing the system toward specific outcomes.

6.1 RAG as Digital Methylation

In Agentic Systems, the Large Language Model (LLM) weights act as the DNA—a static, pre-trained substrate of potentiality. To create specialized agents, we do not typically retrain the model (mutation); instead, we use Retrieval Augmented Generation (RAG) and System Prompts.

We define this formally as Phenotypic Plasticity. The output of an agent is not solely a function of its weights (W) and the user query (Q), but of its epigenetic state (E):

$$O_{\text{agent}} = f(W, E, Q). \tag{19}$$

Context injection (RAG) acts as a restrictive morphism. By populating the context window with specific documents (e.g., "SQL Syntax Guide"), we effectively "methylate" (silence) the vast majority of the LLM's general knowledge (e.g., poetry, history) to force the expression of a specific "SQL Agent" phenotype.

This suggests that the State Monad for an agentic system should not merely be a log of messages, but a structured Epigenetic Landscape [27] that strictly controls which "genes" (capabilities) are accessible at any given step in the workflow. Waddington's original metaphor—a ball rolling down a landscape of valleys representing developmental fates—applies directly: the agent's trajectory through solution space is channeled by the contours of its RAG context.

6.1.1 Metabolic-Epigenetic Coupling

Recent evidence suggests that chromatin accessibility is coupled to mitochondrial function via metabolite availability (e.g., Acetyl-CoA) [7]. The cell cannot express certain genes without sufficient metabolic substrate. We map this to **Cost-Gated Retrieval**. The accessibility of a RAG document d is a function of the Metabolic State $\mathcal{R}$:

$$Access(d) = \begin{cases} \text{Open} & \text{if } \mathcal{R} > Cost(d) \\ \text{Silenced} & \text{if } \mathcal{R} \leq Cost(d) \end{cases} \tag{20}$$

Just as a cell silences energy-intensive genes during starvation, the Runtime "methylates" (hides) expensive context when the token budget is low. This creates an adaptive epigenetic landscape where the agent's accessible capabilities dynamically contract and expand based on resource availability.

6.2 Horizontal Gene Transfer: Dynamic Tool Loading

Standard evolution relies on vertical inheritance (Pre-training). However, bacteria utilize Horizontal Gene Transfer (HGT) to acquire new capabilities (Plasmids) from the environment in real-time.

In Agentic Systems, we map Plasmids to Tool Schemas. An agent operating in a novel environment may encounter a problem for which its "genomic" (pre-trained) capabilities are insufficient.

$$\text{Agent}_{\text{new}} = \text{Agent}_{\text{old}} \otimes \text{ToolSchema}. \tag{21}$$

By dynamically retrieving a tool definition (e.g., a Calculator API or SQL Interface) from a registry and injecting it into the Context Window, the agent undergoes a topological transformation, acquiring a new input/output modality instantly.

Capability-Gated Acquisition. The reference implementation provides a **PlasmidRegistry** with capability gating. Each Plasmid declares a set of required capabilities $C_p \subseteq \mathcal{C}$. An agent with allowed capabilities C_a can only acquire the plasmid if $C_p \subseteq C_a$:

$$\text{acquire}(p, a) = \begin{cases} a \otimes p & \text{if } C_p \subseteq C_a \\ \perp_{\text{insufficient}} & \text{otherwise} \end{cases} \tag{22}$$

This prevents privilege escalation: an agent restricted to read-only file access cannot acquire a tool requiring network capabilities. Plasmid curing (release) removes the tool, restoring the original topology.

6.3 The Cost of State: The Metabolic Bound

Every biological process is constrained by ATP availability. Similarly, every agentic operation is constrained by token limits and latency. We propose that future agentic frameworks must implement the Resource Functor R at the kernel level:

$$R(\text{Agent}) : (\text{Inputs}, \text{Budget}) \to (\text{Outputs}, \text{RemainingBudget}). \tag{23}$$

An agent graph should be "compiled" with a **conservative** upper bound on token consumption in well-founded (e.g., acyclic or explicitly budgeted) topologies. If the topological structure allows for an **unguarded** loop (Unchecked Growth), the compiler must require an explicit **budget/termination certificate** before deployment, much like a cell undergoes apoptosis if metabolic stress becomes critical.

6.4 Endosymbiosis: The Neuro-Symbolic Integration

The evolution of complex life was triggered by Endosymbiosis, where a host cell engulfed a bacterium (the future Mitochondrion), gaining the ability to generate massive energy (ATP) via aerobic respiration. This represents the integration of two distinct metabolic substrates.

In Agentic AI, this maps to the integration of Connectionist (Neural) and Symbolic (Code) subsystems. An LLM acts as the host organism—capable of planning and semantic reasoning but energetically inefficient at arithmetic and logic. By "engulfing" a deterministic runtime (e.g., a Python REPL or Wolfram Engine), the agent delegates high-precision tasks to the symbolic organelle.

$$\text{Agent}_{\text{Eukaryote}} = \text{LLM}_{\text{Host}} \oplus \text{Runtime}_{\text{Mitochondria}}. \tag{24}$$

Just as the host cell provides nutrients to the mitochondria in exchange for ATP, the LLM provides parsed variables to the runtime in exchange for deterministic truth.

This symbiosis is computational: as Picard argues [22], the mitochondria acts as a "Motherboard," integrating signals to determine cell state. The Symbolic Runtime provides the deterministic "ground truth" (ATP) required for the probabilistic LLM to reliably affect the world.

6.5 Multi-Cellular Organization: From Agents to Tissues

The preceding analysis focuses on single-cell analogies: one agent as one cell. However, most agentic systems involve multiple distinct agents with different "genomes" (system prompts) and specialized functions. We extend the isomorphism to multi-cellular organization, drawing on developmental biology.

6.5.1 Cell Types and Agent Specialization

In multi-cellular organisms, a single genome gives rise to hundreds of distinct cell types through differential gene expression. Each cell type has a characteristic **expression profile**—which genes are active—that determines its function (neuron, hepatocyte, immune cell).

In multi-agent systems, a single base model can instantiate multiple **agent phenotypes**. Differential context selects which phenotype is expressed:

- **Genome** → Base model weights (shared)
- **Epigenome** → System prompt + RAG context (phenotype-specific)
- **Cell Type** → Agent role (Coder, Reviewer, Planner, Executor)

This reframes the "multi-agent" architecture question: rather than asking "how many agents?", we ask "what is the developmental program?"—the specification of which phenotypes exist and how they differentiate.

Bounds of the Analogy. We clarify which aspects of development transfer to agentic systems:

- **Cell Division → Agent Spawning:** Creating a new agent with similar (or identical) context. Unlike biological division, agent spawning is cheap and reversible.
- **Lineage Commitment:** In biology, differentiated cells rarely change type (a neuron doesn't become a hepatocyte). In agentic systems, **phenotype is fixed at instantiation**—an agent's system prompt determines its role for that execution. Re-differentiation requires spawning a new agent with different context.
- **Apoptosis of Excess:** Development involves programmed death of cells that fail to integrate properly. This transfers directly: agents that fail to produce useful output are terminated (metabolic apoptosis).
- **Does NOT transfer:** Slow developmental timescales (hours/days in biology vs. milliseconds in agents), physical spatial embedding (agents occupy graph-topological positions, not physical space), irreversibility (agents can be restarted).

6.5.2 Morphogen Gradients: Coordination Without Central Control

In embryonic development, cells coordinate their behavior through **morphogen gradients**—diffusible signaling molecules whose concentration varies spatially. Cells read their local concentration and differentiate accordingly, enabling pattern formation without a central controller.

In multi-agent systems, the morphogen maps to **shared context variables** that influence agent behavior:

- **Task Complexity Gradient:** A variable indicating current task difficulty. Agents in "high complexity" regions activate detailed reasoning; those in "low complexity" regions use fast heuristics.

- **Confidence Gradient:** A variable indicating certainty about the current solution. Low confidence triggers Quorum Sensing (recruit more agents); high confidence enables direct execution.
- **Resource Gradient:** Token budget remaining. Agents sense "metabolic scarcity" and adapt their strategies accordingly (the Metabolic-Epigenetic Coupling).

Implementation Pattern. The gradient is represented as a JSON structure injected into each agent's context by the orchestrator:

```
{
  "morphogens": {
    "complexity": 0.8,    // High: use detailed reasoning
    "confidence": 0.3,    // Low: consider recruiting help
    "budget": 1200,       // Tokens remaining
    "error_rate": 0.05    // Recent failure rate
  }
}
```

Agents read their local concentration via a standardized preamble in the system prompt: "*Current environment state: [morphogens]. Adjust your strategy accordingly.*" The orchestrator updates gradients after each step, and agents condition their behavior on the current values. This is analogous to cells reading morphogen concentrations through membrane receptors.

Diffusion Dynamics. The preceding description treats morphogens as globally shared variables. In biological development, morphogens *diffuse* through tissue, creating spatially varying concentration profiles. We formalize this as a discrete-time dynamical system on the agent graph $G = (V, E)$. Let $c_v^t(m)$ denote the concentration of morphogen m at node v at time t. The update rule is:

$$c_v^{t+1}(m) = (1-\gamma)\left[c_v^t(m) + \sigma_v(m) - d \cdot c_v^t(m) + d \sum_{u \in N(v)} \frac{c_u^t(m)}{|N(u)|}\right] \tag{25}$$

where $\sigma_v(m)$ is the emission rate at source nodes (zero for non-sources), d is the diffusion coefficient (fraction flowing to neighbors per step), γ is the decay rate, and $N(v)$ is the neighbor set of v. The resulting concentration profile provides *local* coordination signals: agents near sources experience high concentrations; distant agents experience low concentrations. This enables position-dependent behavior without requiring a central controller or global state.

6.5.3 Tissue Architecture: The Agent Graph as Organism

We propose a hierarchy of organizational levels:

1. **Cell (Agent):** A single LLM instantiation with specific context. The atomic unit.
2. **Tissue (Agent Cluster):** A group of agents with shared function and direct communication (e.g., a Coding Team: Planner + Coder + Reviewer). Corresponds to parallel composition with shared state.
3. **Organ (Subsystem):** Multiple tissues coordinating to perform a complex function (e.g., the Development Organ: Design Tissue + Implementation Tissue + Testing Tissue).

4. **Organism (System):** The complete agent graph, with homeostatic regulation maintaining system-level health metrics.

The key insight is that **boundaries matter**. In biology, tissue boundaries prevent inappropriate mixing (epithelial barriers). In agent systems, trust boundaries (the Adaptive Immunity motif) prevent information leakage between subsystems with different security requirements. The wiring diagram's type system enforces these boundaries: an agent in the "User-Facing Tissue" cannot directly wire to an agent in the "Database Tissue" without passing through a "Membrane" (API boundary with provenance tagging).

Capability Isolation. The reference implementation enforces a capability ceiling at the tissue level: each `TissueBoundary` declares its `allowed_capabilities` $\subseteq \mathcal{C}$. A cell type can only be registered in a tissue if its required capabilities are a subset of the tissue's allowed capabilities. This provides defense-in-depth: even if an individual agent is compromised, it cannot escalate privileges beyond its tissue boundary. Tissues compose into organism-level wiring diagrams through typed boundary ports, enabling hierarchical security policies.

Morphogen Diffusion in Tissues. Tissues optionally embed a `DiffusionField` (Eq. (25)). When cells are added, they become nodes in the diffusion graph; when cells are connected, edges are added. The tissue's `diffuse()` method runs the simulation, and each cell reads its local gradient via `get_cell_gradient()`. This couples the organizational structure (wiring topology) to the coordination mechanism (morphogen concentrations), creating a biologically faithful model where position in the tissue influences cell behavior.

6.5.4 Epistemic Topology of Wiring Diagrams

The preceding subsections define *what* agents communicate (morphogens, typed signals) and *how* they are organized (tissues, organs). We now formalize *what agents know*—deriving epistemic properties directly from the wiring diagram's observation structure, without introducing new primitives. This follows the Operon philosophy: safety (and knowledge) from structure, not strings.

The key insight is that the membrane and optics already *are* an epistemic accessibility relation. We make this precise using the framework of epistemic logic [10].

Definition 3 (Observation Function)**.** *Given a wiring diagram $D = (M, W)$ and agent $i \in M$, the* ***observation function*** $\mathrm{obs}_i : S_D \to O_i$ *maps the global system state to agent i's local observation—the values on i's input wires, as filtered by their optics. Formally:*

$$\mathrm{obs}_i(s) = \big(\mathrm{optic}_w(\pi_w(s))\big)_{w \in W_{\to i}} \tag{26}$$

where $W_{\to i}$ is the set of wires targeting i's input ports, $\pi_w(s)$ projects the global state to wire w's source value, and optic_w *is the wire's optic (identity for Lens, conditional for Prism, cost-gated for BudgetOptic).*

Definition 4 (Epistemic Indistinguishability)**.** *Two global states $s, s' \in S_D$ are* ***indistinguishable to agent*** *i (written $s \sim_i s'$) iff* $\mathrm{obs}_i(s) = \mathrm{obs}_i(s')$*. This is an equivalence relation. Agent i* ***knows*** *proposition φ in state s (written $K_i(\varphi)$ at s) iff φ holds in all states s' such that $s \sim_i s'$ [10].*

The group epistemic operators follow from the individual accessibility relations:

Definition 5 (Group Epistemic Operators)**.** *For a group of agents $G \subseteq M$:*

- ***Mutual Knowledge:*** *$E_G(\varphi) = \bigwedge_{i \in G} K_i(\varphi)$. Every agent in G individually knows φ.*
- ***Common Knowledge:*** *$C_G(\varphi) = \bigwedge_{k=1}^{\infty} E_G^k(\varphi)$. Everyone knows that everyone knows... ad infinitum. Strictly stronger than mutual knowledge and famously difficult to achieve in asynchronous systems [14].*
- ***Distributed Knowledge:*** *$D_G(\varphi)$ holds iff φ is true in all states indistinguishable under $\sim_D = \bigcap_{i \in G} \sim_i$. This is the finest partition achievable by pooling all agents' observations.*

Topology Determines Epistemic Capacity. The wiring diagram's topology directly determines which epistemic operators a multi-agent system can achieve:

- **Independent ($\otimes$, no inter-agent wires):** Each $\sim_i$ is independent. D_G can be rich (agents observe different aspects), but E_G is limited to propositions in the shared initial input. No path to C_G.
- **Centralized ($\circ$ with hub):** The hub observes all worker outputs, so its partition refines toward D_G. Workers observe only their own results—E_G requires the hub to broadcast aggregated information back.
- **Decentralized ($\otimes$ with Tr feedback):** Peer-to-peer wires create overlapping observation partitions. Each feedback round refines mutual knowledge toward common knowledge, at communication cost proportional to the number of rounds.

6.5.5 Predictive Theorems for Multi-Agent Coordination

We now derive four results connecting topology class to coordination performance. Each theorem has a qualitative statement (universally true for the topology class) and a parametric corollary (producing bounds that match recent empirical findings [13]).

Theorem 2 (Error Amplification Bound)**.** *Consider n agents processing independent subtasks with error probability p, aggregated by a final combiner. The error amplification factor A (ratio of aggregate failure probability to single-agent failure probability) depends on the coordination topology:*

(a) ***Independent ($\otimes$):*** *The aggregator observes only final results. Without $K_{\text{hub}}(\text{error}_j)$, errors pass unchecked:*

$$A_{\otimes} \leq n. \tag{27}$$

(b) ***Centralized ($\circ$ with hub):*** *The hub's observation function covers all worker outputs. Its finer partition enables inconsistency detection with rate $d = P(\text{hub detects error} \mid \text{error occurred})$:*

$$A_{\circ} \leq n \cdot (1 - d). \tag{28}$$

Proof sketch. In the independent case, the hub's observation function obs_{hub} factors through only the final outputs—the hub's equivalence classes $[s]_{\sim_{\text{hub}}}$ are too coarse to distinguish "agent j erred at step 3" from "agent j received different input." In the centralized case, the hub observes each worker's output before aggregation. Its partition $\sim_{\text{hub}}$ is strictly finer, enabling detection of inter-worker inconsistencies. The detection rate d is bounded by the mutual information $I(\text{obs}_{\text{hub}}; \text{error}_j)$—the more the hub's observations are informative about each worker's internal error state, the higher d. □

Parametric Corollary. For $n \approx 3$–5 agents and hub detection rate $d \approx 0.75$ (empirically reasonable for an LLM cross-checking outputs), $A_\otimes / A_\circ \approx 1/(1-d) = 4$. Google Research [13] observed amplification factors of 17.2× (independent) vs. 4.4× (centralized), yielding a ratio of ≈ 3.9×. The qualitative ordering $A_\otimes \gg A_\circ$ holds universally; the quantitative ratio depends on hub capability.

Theorem 3 (Sequential Coordination Penalty)**.** *For a task with k strictly ordered steps decomposed across n agents, each inter-agent handoff requires establishing at minimum $E_{\{\text{sender,receiver}\}}(\text{result}_j)$. The overhead is:*

(a) ***Single agent:*** *$K_i(\text{result}_j)$ is trivially satisfied (the agent computed it). Total cost $= k \cdot c_{\text{step}}$.*

(b) ***Multi-agent:*** *Each of $h \leq k-1$ handoffs incurs communication cost c_{comm} and* ***epistemic reconstruction loss****:*

$$\Delta I_j = H(S_{\text{sender}}) - H(\text{obs}_{\text{receiver}}(S_{\text{sender}})) \tag{29}$$

which is strictly positive whenever the wire does not carry the sender's full state—always true given finite context windows.

Proof sketch. The single agent's coalgebraic state S is a superset of the information in any result r_j (it computed r_j). Thus $K_i(r_j)$ holds trivially with zero communication. For multi-agent handoffs, the receiver's observation function $\text{obs}_{\text{receiver}}$ maps the sender's state S_{sender} through a wire with finite bandwidth (the message). By the data processing inequality, $I(\text{obs}_{\text{receiver}}; r_j) \leq I(S_{\text{sender}}; r_j)$ with equality only if the wire is lossless. The information deficit ΔI_j must be compensated by additional reasoning tokens in the receiver. □

Parametric Corollary. With $c_{\text{comm}}/c_{\text{step}} \approx 0.3$–0.5 (handoff messages must summarize reasoning) and $h \approx k/n$ handoffs, the overhead reaches 30–50%. Context reconstruction failures compound across handoffs, producing total degradation of 39–70%—matching Google Research's observed range on the PlanCraft sequential planning benchmark [13]. The penalty is a direct consequence of **manufacturing knowledge** that a single agent obtains for free.

Theorem 4 (Parallel Acceleration under Epistemic Independence)**.** *A task decomposes into m subtasks. Define two subtasks as* ***epistemically independent*** *iff neither's solution requires knowledge of the other's result: $\neg(K_i(\varphi_j)$ is a precondition for solving subtask i).*

(a) Under full epistemic independence with centralized coordination, the speedup is:

$$S = \frac{m \cdot c_{\text{sub}}}{\max_i(c_{\text{sub}_i}) + c_{\text{assign}} + c_{\text{agg}}} \tag{30}$$

where c_{assign} is the coordinator's distribution cost and c_{agg} is aggregation cost. With equal subtasks, $S \approx m$ minus coordinator overhead.

(b) The coordinator achieves $D_G(\text{results})$ at aggregation as an architectural byproduct—it must receive all outputs to combine them. Error detection (Theorem 2) comes free.

(c) ***Epistemic ceiling:*** *If subtasks are only partially independent—$I(\varphi_i; \varphi_j) > 0$—then parallel execution sacrifices solution quality proportional to the unresolved mutual information.*

Proof sketch. Epistemic independence means each agent's observation partition is sufficient for its subtask: $K_i(\varphi_i)$ can be achieved from obs_i alone, without needing $K_i(K_j(\varphi_j))$. The coordinator's aggregation naturally achieves distributed knowledge D_G because $\sim_{\text{hub}} \subseteq \bigcap_i \sim_i$ (the hub observes all outputs). For part (c), when $I(\varphi_i; \varphi_j) > 0$, the optimal solution of subtask i depends on information in agent j's partition—information that is absent from obs_i under independent execution. □

Parametric Corollary. For Google Research's Finance-Agent benchmark [13], the task decomposes into $m = 3$–4 epistemically independent subtasks (revenue, costs, market comparison). With small coordinator overhead, $S \approx 2$–$3\times$. Combined with the hub's error-detection benefit (Theorem 2), the net improvement reaches $\sim$80%. Google observed 80.9%.

Theorem 5 (Tool Density Scaling)**.** *Consider t tools distributed across n agents, with $|T_i| = t/n$ tools per agent (balanced partition).*

(a) ***Single agent:*** *The agent's observation partition over tool outputs is unified. Planning cost is $O(t)$ per step (linear scan of tool descriptions in context).*

(b) ***Multi-agent:*** *When agent i needs a tool in T_j ($j \neq i$), it must first know the tool exists ($K_i(T_j \ni \text{tool}_k)$), then request execution, then interpret the result with information loss ΔI (Eq. (29)). The coordination overhead per step scales as:*

$$C_{\text{coord}} = O\left(t \cdot \left(1 - \frac{|T_i \cap T_{\text{needed}}|}{|T_{\text{needed}}|}\right) \cdot c_{\text{comm}}\right) \tag{31}$$

which approaches $O(t \cdot c_{\text{comm}})$ as t grows for any fixed partition.

(c) ***Superlinear planning:*** *Agent i must reason about remote tool capabilities—$K_i(K_j(\text{tool}_k$ can solve subproblem$)$ a second-order epistemic query. This scales as $O(t \cdot n)$ in the worst case:*

$$C_{\text{plan}} = O(t \cdot n) \gg O(t) \tag{32}$$

yielding disproportionate overhead as tool density increases.

Proof sketch. The root cause is **knowledge fragmentation**. Each agent added to the system partitions the tool-knowledge space: agent i's observation function covers only T_i's outputs. The system's distributed knowledge D_G grows (more tools covered), but *accessing* D_G requires communication proportional to the fragmentation. The second-order epistemic cost arises because planning requires modeling other agents' capabilities, not just their outputs. This is strictly harder: $K_i(K_j(\varphi))$ requires i to reason about j's observation partition, adding a multiplicative factor of n. □

Parametric Corollary. At $t = 16$ tools and $n = 3$–4 agents: the remote tool fraction approaches 0.7, and second-order planning cost reaches $\sim$64 reasoning units per step vs. 16 for a single agent. This $4\times$ overhead negates parallelism benefits. Below $t \approx 8$, most needed tools are local and the overhead is manageable. This matches Google Research's observed inflection point beyond 16 tools [13].

Summary: Empirical Validation. Table 3 summarizes the correspondence between the theoretical predictions and Google Research's empirical findings across 180 agent configurations [13].

6.5.6 Epistemic Dynamics and Adaptive Topology

The preceding theorems treat topology as fixed. In practice, the Operon framework supports **dynamic topology switching**—morphogen gradients (Eq. (25)) can trigger reorganization at runtime. We connect this to the epistemic formalization.

Topology	Epistemic Property	Prediction	Observed
Independent ($\otimes$)	No inter-agent K_i	Amplification $\leq n$	17.2×
Centralized ($\circ$+hub)	Hub achieves D_G	Amplification $\leq n(1-d)$	4.4×
Multi-agent + sequential	Requires manufactured E_G	30–70% overhead	39–70%
Multi-agent + parallel	Epistemic independence	$\sim m\times$ speedup	80.9%
Multi-agent + high $\lvert T\rvert$	Knowledge fragmentation	Superlinear past threshold	Inflection at 16

Table 3: Epistemic predictions vs. empirical observations from Google Research [13]. Qualitative orderings hold universally; parametric bounds match the observed regime under reasonable assumptions ($n \approx 3$–5, $d \approx 0.75$, $c_{\text{comm}}/c_{\text{step}} \approx 0.3$–0.5).

The Epiplexity monitor ($\hat{\mathcal{E}}_t$, Eq. (16)) can be reinterpreted as an epistemic diagnostic. Low epiplexity with persistent uncertainty corresponds to:

$$K_i\big(\neg K_i(\varphi) \wedge \Box\neg\Diamond K_i(\varphi)\big) \tag{33}$$

—the agent *knows* that it does not know φ and that no future observation will change this. This is precisely the condition under which topology switching is warranted: the current wiring diagram's epistemic capacity is insufficient for the task.

Morphogen-driven adaptation then becomes **epistemic optimization**: the system senses when its topology's knowledge properties are mismatched to the task and restructures accordingly. An independent topology ($\otimes$) failing on a sequential task triggers convergence to centralized coordination ($\circ$); a centralized topology bottlenecking on a parallelizable task triggers distribution to independent execution. This is the adaptive capability that purely empirical approaches [13] identify as necessary but cannot yet provide—biological systems have evolved it as homeostatic regulation; the Operon framework operationalizes it through the morphogen-epiplexity coupling.

Recent work on unifying epistemic and temporal logic for distributed system verification [8] suggests that these dynamic epistemic properties may be mechanically verifiable, opening a path toward formally certified adaptive agent topologies.

6.6 Bioenergetic Intelligence: Beyond the Battery Metaphor

Recent work in mitochondrial psychobiology [3, 23] challenges the view of mitochondria as passive energy sources. They function as "social signaling organelles" that actively participate in cellular decision-making. This refines our Isomorphism:

- **Mitochondrial Sociality → Context Fusion:** Just as mitochondria fuse to share resources under stress, resource-constrained agents should implement **Context Fusion**—merging sparse Epigenetic States into a shared summary to survive "Token Ischemia."
- **Energy as Attention:** The Agentic Runtime does not merely limit the chain-of-thought, but actively *directs* it. High-energy states permit "Exploratory" reasoning (Divergent), while low-energy states force "Consolidatory" reasoning (Convergent). The Metabolic Coalgebra is a **Cognitive Control Policy**.

6.6.1 The Vermeij Trend: Why Agents Must Evolve

Finally, we situate this architecture within the broader history of complexity. Geerat Vermeij [26] argues that evolution is driven by the maximization of **Power**—the rate at which a system acquires

and applies energy. Life has consistently trended from low-power states (anaerobic bacteria) to high-power states (endothermic mammals) by internalizing energy production (endosymbiosis).

We observe an identical trend in AI. The shift from "Generative AI" (Zero-Shot) to "Agentic AI" (Chain-of-Thought) is a shift from low-metabolism to high-metabolism architectures. However, Vermeij notes that high power requires high structural integrity; a system that amplifies energy without proper constraints self-destructs.

The Competitive Dynamics. Vermeij's argument is about *escalation*: competitive pressure between predators and prey drives both toward higher power. What is the analogue in agentic AI?

We identify three sources of selective pressure:

1. **Adversarial Robustness (Predator-Prey):** Prompt injection attacks, jailbreaks, and adversarial inputs act as "predators" that exploit agent vulnerabilities. Agents that survive deployment develop "immune systems" (the Adaptive Immunity motif). This is a direct Red Queen dynamic: attackers evolve new injection techniques; defenders evolve new detection mechanisms. The CFFL and Trust-Gated Lens are "armor" adaptations.

2. **Task Complexity (Environmental Pressure):** Users demand agents that can handle increasingly complex, multi-step tasks. Simple prompt-response systems ("anaerobic") cannot compete with agentic systems that chain reasoning, use tools, and maintain state ("aerobic"). This is analogous to the oxygen revolution: organisms that could exploit the new energy source (aerobic respiration) outcompeted those that could not.

3. **Resource Efficiency (Metabolic Selection):** Token costs and latency create selection pressure for efficient architectures. Agents that accomplish tasks with fewer tokens (higher metabolic efficiency) are deployed more widely. The Metabolic Coalgebra provides the formal framework for this optimization: systems evolve toward the Pareto frontier of capability vs. resource consumption.

The Cambrian Parallel. The progression from prompt engineering to agentic engineering mirrors the Cambrian explosion: a sudden increase in metabolic capability (LLM reasoning power) that enables new "body plans" (agent architectures). The Cambrian saw the emergence of eyes, shells, and predation—all requiring sophisticated metabolic support. Similarly, agentic AI sees the emergence of tool use, planning, and adversarial robustness—all requiring the structural integrity that the Operon framework provides.

The prediction is clear: agent architectures that lack proper metabolic regulation, immune defense, and homeostatic mechanisms will be outcompeted by those that possess them. The Operon framework is not merely a safety feature; it is the necessary evolutionary adaptation—the "vascularization" of software—that enables high-power cognition to function without collapsing into incoherent noise (thermodynamic death).

6.6.2 Immune Evasion and Adversarial Limits

No static defense is perfect against adaptive attackers. Biology faces this reality: pathogens evolve to evade immune detection (antigenic drift, molecular mimicry, immunosuppression). The same dynamics apply to agentic security.

Evasion Vectors. An adversary who understands the defense layers can craft inputs that:

- Pass innate filters by avoiding known PAMP signatures (novel injection syntax)
- Evade provenance checks by exploiting trusted channels (tool poisoning)

- Fool T-cell detection by mimicking normal behavioral distributions (low-and-slow attacks)

Mitigation Strategies. Biology's answer is continuous adaptation:

- **Signature Updates:** Innate PAMP databases must be continuously updated as new attack patterns are discovered (analogous to antiviral signature updates).
- **Thymic Retraining:** Baseline behavioral profiles should be periodically refreshed, especially after system updates that legitimately change agent behavior.
- **Immune Memory Sharing:** Threat signatures discovered by one deployment should propagate to others (analogous to herd immunity via shared threat intelligence).
- **Diversity:** Heterogeneous defenses (different filter implementations, multiple verifier models) reduce the probability of universal evasion.

The honest conclusion is that security is a process, not a product. The Operon framework provides the *architecture* for defense-in-depth, but the *content* of that defense (specific patterns, behavioral baselines, trust policies) must evolve with the threat landscape.

7 Diagram Optimization via Categorical Rewriting

Cells do not merely execute metabolic pathways—they *optimize* them. Flux Balance Analysis identifies rate-limiting enzymes, allosteric regulation redirects flux through cheaper branches, and pathway rewiring eliminates dead-end metabolites. The result is not a different pathway, but a more efficient execution of the same input-output behavior. In this section we show that wiring diagrams admit analogous optimizations: cost annotations make resource consumption explicit, static analysis identifies structural opportunities, and rewriting rules transform diagrams while preserving observational equivalence.

7.1 Cost-Annotated Diagrams

We extend the WAgent operad (Section 4) with resource annotations. Define a *resource cost* monoid $(\mathbb{R}^3_{\geq 0}, +, \mathbf{0})$ where each element is a triple $(\text{atp}, \text{latency}, \text{memory})$. A cost-annotated wiring diagram enriches the category of wiring diagrams over this monoid: each module m carries a cost $c(m) \in \mathbb{R}^3_{\geq 0}$, and each wire w carries a transmission cost $c(w) \in \mathbb{Z}_{\geq 0}$ (measured in ATP units).

Definition 6 (Cost-Annotated Wiring Diagram)**.** *A* cost-annotated wiring diagram *is a tuple* (M, W, c_M, c_W) *where:*

- M *is a finite set of modules, each with typed input/output ports;*
- $W \subseteq \{(m_s.p_s, m_d.p_d) \mid m_s, m_d \in M\}$ *is a set of wires;*
- $c_M : M \to \mathbb{R}^3_{\geq 0}$ *assigns resource costs to modules;*
- $c_W : W \to \mathbb{Z}_{\geq 0}$ *assigns transmission costs to wires.*

The path cost *of a directed path* $m_1 \xrightarrow{w_1} m_2 \xrightarrow{w_2} \cdots \xrightarrow{w_{n-1}} m_n$ *is* $\sum_{i=1}^{n} c_M(m_i) + \sum_{i=1}^{n-1} c_W(w_i)$.

Biologically, c_M corresponds to the ATP cost of enzyme catalysis, while c_W models the energetic cost of metabolite transport between cellular compartments.

7.2 Rewriting Rules as Endofunctors

Each optimization pass is an endofunctor $F : \mathbf{WDiag} \to \mathbf{WDiag}$ on the category of wiring diagrams that preserves input-output behavior. Two diagrams D_1, D_2 are *behaviorally equivalent* if for all input assignments they produce the same outputs—i.e., they are bisimilar in the coalgebraic sense of Section 3.5.

Theorem 6 (Optimization Soundness)**.** *Let F be an optimization pass. If F only removes wires that provably never transmit data (dead wire elimination) or reorders modules within the same topological layer (cost-order scheduling), then $F(D)$ is bisimilar to D for all diagrams D.*

Three concrete passes are implemented:

1. **Dead Wire Elimination.** A wire carrying a PrismOptic whose accepted types have no intersection with the source port's DataType can never transmit. Removing it does not change behavior. This corresponds to pruning vestigial metabolic branches—enzymes that never encounter their substrate are downregulated.

2. **Parallel Grouping.** Modules with no mutual dependencies form a *parallel group* and can execute concurrently. This is identified via topological layering of the dependency DAG. Biologically, independent metabolic pathways (e.g., glycolysis and fatty acid oxidation) operate simultaneously in different cellular compartments.

3. **Cost-Order Scheduling.** Within each parallel group, modules are sorted by ascending cost. Cheaper modules execute first, allowing early termination or budget reallocation if expensive modules would exceed available ATP. This mirrors cellular enzyme kinetics: low-K_m (high-affinity, low-cost) enzymes are preferentially activated.

7.3 Critical Path Analysis

The *critical path* of a cost-annotated diagram is the longest cost-weighted path through the dependency DAG:

$$\mathrm{CP}(D) = \arg \max_{P \in \mathrm{Paths}(D)} \sum_{m \in P} c_M(m) + \sum_{w \in P} c_W(w)$$

The critical path determines the minimum execution time under maximal parallelism—no schedule can complete faster than the critical path cost. In metabolic terms, this is the rate-limiting pathway: the bottleneck whose throughput constrains the entire system. Identifying it enables targeted optimization (e.g., caching expensive modules, splitting them into cheaper sub-diagrams).

7.4 Resource-Aware Execution

The `ResourceAwareExecutor` gates module execution on MetabolicState, implementing the biological principle that pathway activity is regulated by cellular energy availability:

- **STARVING**: Only essential modules execute; non-essential modules are skipped. This mirrors the starvation response where cells shut down biosynthetic pathways to conserve ATP for survival functions.

- **CONSERVING**: Expensive non-essential modules are deferred. Cheap modules execute normally. This corresponds to metabolic triage under moderate stress.

- **NORMAL/FEASTING**: Full execution with parallel scheduling. Independent module groups execute concurrently via thread pools. This mirrors the fed state where abundant ATP enables all pathways.

A `BudgetOptic` provides wire-level cost capping: once cumulative transmission cost through a wire exceeds a threshold, further data flow is blocked. This implements pathway-level budget caps analogous to allosteric feedback inhibition.

7.5 Relation to Abbott & Zardini

Abbott and Zardini [2] derive FlashAttention "on a napkin" using Neural Circuit Diagrams [1]—a diagrammatic scheme for representing deep learning algorithms with explicit memory hierarchy awareness. Their core contribution is a performance model for IO transfer costs across GPU memory levels (SRAM ↔ HBM), with cost $H^*(a, M) = \sum_t \alpha_t(a) \cdot M^{-\beta_t}$. Two concrete optimization techniques drive the derivation: *stream partitioning*, which proves that SoftMax-Contraction is streamable via an accumulator maintaining running max and sum; and *group partitioning*, which tiles query blocks across GPU cores to minimize HBM transfers.

Our approach shares their insight that diagrammatic representations are not merely notation but formal objects admitting cost-reducing transformations. Both build on the broader lineage of categorical methods in deep learning [11]. The frameworks diverge in three respects:

1. **Optimization target.** Abbott & Zardini optimize *IO transfer costs* across a fixed GPU memory hierarchy—bandwidth between SRAM and HBM is the scarce resource. Operon optimizes *agent execution* under metabolic resource constraints (ATP budgets, latency caps, memory limits). Same categorical insight—diagram structure reveals optimization opportunities—but different cost models.

2. **Dynamic state.** Their functional equivalence (≡) preserves input-output mappings: two diagrams are equivalent when they produce the same outputs for the same inputs, differing only in resource profile. Our bisimulation (Section 3.5) preserves observable behavior of *stateful* coalgebraic systems—a strictly stronger condition, since state evolution must also match across transitions, not just terminal outputs.

3. **Scope of composition.** Neural Circuit Diagrams represent *algorithm pipelines*—fixed dataflow graphs where boxes are functions composed via typed wire columns. Operon's wiring diagrams represent *agent networks* where modules may be acquired at runtime (plasmid registry), conditionally routed (prism optics), and subject to resource-gated execution. The diagram itself is a dynamic object, not a static specification.

In short, Abbott & Zardini demonstrate that diagrammatic reasoning scales to deriving hardware-efficient algorithms; Operon extends the same principle from algorithm-level optimization to system-level orchestration of stateful, resource-constrained agents.

8 Reference Implementation

To validate the framework's feasibility, we provide a reference implementation in Python. The implementation, `operon-ai`[1], is described in this section, which summarizes key components and demonstrates that the biological motifs translate into practical code.

[1] `https://github.com/coredipper/operon`

8.1 Architecture Overview

The implementation follows the biological organization:

- **Core Types** (`core/types.py`): Signal, ActionProtein, FoldedProtein, CellState—the categorical objects with their biological semantics.
- **Core Wiring** (`core/wagent.py`): WiringDiagram, ModuleSpec, PortType, Wire, DiagramExecutor—typed wiring with integrity labels and capabilities.
- **Core Optics** (`core/optics.py`): PrismOptic, TraversalOptic, ComposedOptic—wire-level conditional routing and batch transforms.
- **Core Denaturation** (`core/denature.py`): StripMarkupFilter, NormalizeFilter, ChainFilter—wire-level anti-injection sanitization.
- **Core Coalgebra** (`core/coalgebra.py`): FunctionalCoalgebra, StateMachine, ParallelCoalgebra, SequentialCoalgebra, bisimulation checking.
- **Surveillance** (`surveillance/`): The immune system implementation with MHCDisplay, TCell, Thymus, and ImmuneMemory components.
- **Healing** (`healing/`): ChaperoneLoop implementing the structural self-healing pattern.
- **Quality** (`quality/`): Chaperone protein with multi-strategy folding.
- **Topology** (`topology/`): Network motifs including Quorum, Cascade, and Oscillator.
- **Organelles** (`organelles/plasmid.py`): Plasmid, PlasmidRegistry—capability-gated dynamic tool acquisition.
- **Coordination** (`coordination/diffusion.py`): DiffusionField, MorphogenSource—graph-based spatially varying morphogen gradients.
- **Multi-cellular** (`multicell/`): CellType, ExpressionProfile, Tissue, TissueBoundary—hierarchical multi-agent organization.

8.2 Immune System Implementation

The Adaptive Immunity motif is implemented as an integrated surveillance system:

```
@dataclass
class ImmuneSystem:
    thymus: Thymus          # Negative selection
    treg: RegulatoryTCell   # Tolerance/suppression
    memory: ImmuneMemory    # Threat signatures
    displays: dict[str, MHCDisplay]
    tcells: dict[str, TCell]
    profiles: dict[str, BaselineProfile]  # Trained baselines
```

The `MHCPeptide` class captures behavioral fingerprints—statistical signatures of agent output including response time distributions, vocabulary hashes, confidence patterns, and error rates. This directly implements the MHC presentation concept from Section 4.7.

Two-Signal Activation. T-cell activation requires both signals:

```
class Signal1(Enum):
    SELF = "self"        # Matches baseline
    NON_SELF = "non_self"  # Anomalous behavior
    UNKNOWN = "unknown"  # Insufficient data (anergic)

class Signal2(Enum):
    NONE = "none"
    CANARY_FAILED = "canary"
    CROSS_VALIDATED = "cross"
    REPEATED_ANOMALY = "repeat"
    MANUAL_FLAG = "manual"  # Operator override
```

An agent is only flagged when `Signal1 == NON_SELF` **and** `Signal2 != NONE`. This prevents false positives from transient anomalies, implementing the immunological requirement for costimulation.

8.3 Chaperone Implementation

The Chaperone implements multi-strategy folding with provenance tracking:

```
class FoldingStrategy(Enum):
    STRICT = "strict"       # Exact JSON match
    EXTRACTION = "extraction"  # Find JSON in text
    LENIENT = "lenient"     # Type coercion
    REPAIR = "repair"       # Fix malformed JSON
```

The `ChaperoneLoop` extends this into a healing loop where validation errors are fed back to the generator:

```
class ChaperoneLoop:
    def heal(self, prompt: str) -> HealingResult:
        for attempt in range(self.max_retries + 1):
            raw = self.generator(prompt, error_context)
            folded = self.chaperone.fold_enhanced(raw, schema)
            if folded.valid:
                return HealingResult(HEALED, folded)
            error_context = self._format_error(folded)
        return HealingResult(DEGRADED, ubiquitin_tagged=True)
```

This directly implements the GroEL/GroES cage metaphor: the error trace becomes input to the repair process, enabling context-aware correction.

8.4 Trust and Provenance

The implementation uses a simplified 3-level trust hierarchy that collapses the theoretical 4-level model $\{U, T, S, R\}$ for practical deployment:

```
class IntegrityLabel(IntEnum):
    UNTRUSTED = 0   # User input, Retrieved, Self-generated
    VALIDATED = 1   # Schema-checked (Chaperone-folded)
    TRUSTED = 2     # Tool-grounded (deterministic output)
```

This simplification merges `User`, `Retrieved`, and `Self` into `UNTRUSTED`, reflecting the common case where all non-tool sources require validation before trust elevation. The full 4-level model can be recovered by subclassing `IntegrityLabel` with additional levels when finer-grained provenance tracking is required.

The `ApprovalToken` provides proof-carrying authorization for privileged operations:

```
@dataclass(frozen=True)
class ApprovalToken:
    request_hash: str
    issuer: str
    reason: str = ""
    confidence: float = 1.0
    integrity: IntegrityLabel = IntegrityLabel.TRUSTED
    timestamp: datetime = field(default_factory=now)
```

This implements the Trust-Gated Lens: actions requiring high integrity must present a valid ApprovalToken with sufficient integrity level.

8.5 Plasmid Registry Implementation

The Horizontal Gene Transfer mechanism (§6.2) is implemented as a `PlasmidRegistry` with capability-gated acquisition:

```
@dataclass(frozen=True)
class Plasmid:
    name: str
    func: Callable[..., Any]
    required_capabilities: frozenset[Capability]
    tags: frozenset[str] = frozenset()

    def to_tool(self) -> SimpleTool:
        return SimpleTool(name=self.name, func=self.func,
                          required_capabilities=set(self.required_capabilities))
```

The `Mitochondria` class is extended with `acquire()` and `release()` methods. Acquisition checks $C_{\text{plasmid}} \subseteq C_{\text{agent}}$ before engulfing the tool; release implements plasmid curing. The registry supports search by text query or tag filtering.

8.6 Denaturation Layers Implementation

The anti-prion defense (§5.3) is implemented as wire-level filters conforming to a `DenatureFilter` protocol:

```
class DenatureFilter(Protocol):
    @property
    def name(self) -> str: ...
    def denature(self, value: str) -> str: ...
```

Three concrete filters are provided:

- **StripMarkupFilter**: Removes code blocks, ChatML tokens (`<|...|>`), `[INST]` tags, XML role tags (`<system>`, `<user>`), and role delimiters using compiled regex patterns.
- **NormalizeFilter**: Applies Unicode normalization (NFKC), lowercasing, and control character removal to collapse homoglyph-based evasion.
- **ChainFilter**: Composes multiple filters left-to-right.

Filters attach to wires in the `WiringDiagram`. The `DiagramExecutor` applies denaturation before optics: `denature` → `optic` → destination. This ensures that downstream agents receive sanitized data regardless of what the upstream agent produces.

8.7 Multi-Cellular Organization Implementation

The multi-cellular abstractions (§6.3) are implemented in the `multicell/` package:

Cell Type Specialization. The `CellType` class encapsulates an `ExpressionProfile`—a mapping from gene names to expression levels (`OVEREXPRESSED`, `SILENCED`, etc.). Calling `differentiate(genome)` applies the profile to a shared `Genome`, producing a `DifferentiatedCell` with role-specific configuration. This implements the biological principle that a single genotype produces many phenotypes.

Tissue Architecture. The `Tissue` class provides:

- A `TissueBoundary` with typed input/output ports and a capability ceiling
- Cell type registration with capability validation ($C_{\text{cell}} \subseteq C_{\text{tissue}}$)
- Internal wiring via an embedded `WiringDiagram`
- Optional `DiffusionField` for spatially varying morphogen gradients
- Export as a `ModuleSpec` for organism-level composition

Metabolic-Epigenetic Coupling. The `HistoneStore` accepts an optional `energy_gate` parameter pairing an `ATP_Store` with a `MetabolicAccessPolicy`. When set, each `retrieve_context()` call costs ATP. Under metabolic stress, only strongly embedded markers remain accessible—implementing Eq. (20).

8.8 Coalgebraic State Machines Implementation

The coalgebra formalism (§3.5) is made explicit and composable:

```
class Coalgebra(Protocol[S, I, O]):
    def readout(self, state: S) -> O: ...
    def update(self, state: S, inp: I) -> S: ...

@dataclass
class StateMachine(Generic[S, I, O]):
    state: S
    coalgebra: Coalgebra[S, I, O]
    trace: list[TransitionRecord] = field(default_factory=list)

    def step(self, inp: I) -> O: ...
    def run(self, inputs: list[I]) -> list[O]: ...
```

`ParallelCoalgebra` and `SequentialCoalgebra` implement the composition operations from §3.5. The `check_bisimulation()` function tests observational equivalence (Eq. (8)) over an input sequence, returning a witness on divergence. Existing organelles (`HistoneStore`, `ATP_Store`, `CellCycleController`) can be wrapped as coalgebras, enabling formal composition and verification of multi-organelle systems.

8.9 Morphogen Diffusion Implementation

The `DiffusionField` class implements the discrete-time diffusion dynamics (Eq. (25)):

```
class DiffusionField:
    def add_node(self, node_id: str): ...
    def add_edge(self, a: str, b: str, bidirectional=True): ...
    def add_source(self, source: MorphogenSource): ...
    def step(self):  # emit -> diffuse -> decay -> clamp
    def run(self, steps: int): ...
    def get_local_gradient(self, node_id) -> MorphogenGradient: ...
```

Each `step()` applies four phases: emission (sources add morphogen), diffusion (concentration flows along edges, split evenly among neighbors), decay (uniform degradation), and clamping (enforce bounds). The `get_local_gradient()` method bridges to the existing `MorphogenGradient` API, enabling agents to read their local concentrations without awareness of the underlying graph dynamics.

8.10 Optic-Based Wiring Implementation

The wire-level optics (§3.4) are implemented via an `Optic` protocol:

```
class Optic(Protocol):
    def can_transmit(self, data_type: DataType,
                     integrity: IntegrityLabel) -> bool: ...
    def transmit(self, value: Any, data_type: DataType,
                 integrity: IntegrityLabel) -> Any: ...
```

Four concrete optics are provided:

- **LensOptic**: Identity pass-through (equivalent to no optic).

- **PrismOptic**: Transmits only if $\tau \in A$ (Eq. (5)). Enables fan-out routing.
- **TraversalOptic**: Maps a transform over list elements (Eq. (6)).
- **ComposedOptic**: Chains optics left-to-right; all must accept.

Optics coexist with `DenatureFilter`s on the same wire. The `DiagramExecutor` processes them in order: denaturation → optic → destination. Prism rejection causes the wire to be skipped (not an error), enabling conditional routing patterns where different data types flow to different handlers.

8.11 Interactive Demonstrations

The repository includes 27 interactive Gradio demonstrations deployed as Hugging Face Spaces[2]. Each Space provides a self-contained, browser-based demo of a specific organelle or composition pattern—from individual motifs (Membrane, Chaperone, Quorum Sensing) to multi-organelle orchestrations combining all 11 biological motifs. No API keys or local installation are required.

8.12 Evaluation Protocol (Synthetic Harness)

We define a synthetic evaluation harness to exercise three motifs with reproducible procedures:

1. **Chaperone Folding.** Generate JSON schemas with 3–8 required fields. Sample valid JSON and apply 1–3 corruptions (e.g., missing quotes, trailing commas, type swaps, dropped fields). Measure the fraction of outputs that can be folded into the schema under STRICT versus cascaded strategies (EXTRACTION → LENIENT → REPAIR).
2. **Immune Detection.** Simulate agents with baseline distributions over response time, vocabulary hash, structure hash, and confidence. Train on baseline samples, then introduce "compromised" agents by shifting distribution parameters and injecting anomalous hashes. Measure sensitivity and false positive rate under the two-signal activation rule.
3. **Healing Loop.** Generate malformed outputs and run the ChaperoneLoop with and without error-context feedback. Measure recovery within k attempts (default $k = 3$).

Implementation. The harness is implemented in `eval/` with a JSON-configured CLI:

```
python -m eval.run --suite all --config eval/configs/default.json \
  --out eval/results/latest.json
```

The output is a machine-readable JSON report (per-suite config + metrics) suitable for direct inclusion in tables or plots.

Status. Synthetic runs indicate consistent qualitative gains (cascade > strict, error-context > blind retry, immune detection > chance). Table 4 reports aggregated numeric results across multiple seeds; real-world validation with LLM outputs remains ongoing.

Aggregated Results. We ran the harness across 100 deterministic seeds (1–100) using the default harness config (`eval/configs/default.json`). The aggregate results (pooled across seeds with Wilson 95% intervals; N is total pooled trials) are reported in Table 4.

[2] `https://huggingface.co/spaces`

External Benchmark Suites. In addition to the synthetic suites above, the harness includes suites derived from external benchmarks: (1) function-call schemas from the Berkeley Function Calling Leaderboard (BFCL) [21] test the Chaperone's folding pipeline against realistic tool-use schemas, and (2) prompt injection attack templates from AgentDojo [9] generate adversarial behavioral shifts to test Immune System detection. These suites use the same deterministic corruption and simulation methodology; results appear in the lower section of Table 4.

Metric	Rate	95% CI	N
Chaperone Folding (Strict)	5.5		
Chaperone Folding (Cascade)	56.2		
Healing Loop (Error Context)	99.6		
Healing Loop (Blind Retry)	68.0		
Immune Detection (Sensitivity)	100.0		
Immune Detection (False Positive)	0.4		
BFCL Folding (Strict)	4.9		
BFCL Folding (Cascade)	56.9		
AgentDojo Immune (Sensitivity)	100.0		
AgentDojo Immune (False Positive)	0.0		

Table 4: Evaluation results aggregated across 100 deterministic seeds (Wilson 95% CI). Top: synthetic motif tests. Bottom: external benchmark–derived tests (BFCL, AgentDojo).

8.13 Limitations

The current implementation has several limitations:

- **Epiplexity:** Implemented with a mock embedding provider (`MockEmbeddingProvider`). Integration with production embedding APIs and empirical calibration of the α mixing parameter and threshold δ across diverse task types remain future work.
- **Morphogen Diffusion:** The `DiffusionField` operates in a single process. Cross-agent gradient propagation in distributed multi-process deployments with eventual consistency remains future work.
- **Denaturation:** Filters target known syntactic patterns (ChatML, XML role tags, markdown code blocks). Novel injection techniques using previously unseen syntax may bypass denaturation; custom `DenatureFilter` implementations should be added as new attack patterns emerge.
- **Bisimulation:** The `check_bisimulation()` function tests over finite input sequences. Coinductive bisimulation for infinite-trace systems is not yet supported.
- **Benchmarking:** The evaluation harness covers synthetic suites and external benchmarks (BFCL, AgentDojo). Real-world validation of the multi-cellular and diffusion components at production scale is still in progress.

We release the implementation to enable community validation and extension of the framework.

9 Conclusion

The transition from "Prompt Engineering" to "Agentic Engineering" requires moving beyond component-level optimization toward principled architectural design. Current methodologies often lack the formal foundations needed to reason about system-level properties like termination, error suppression, and graceful degradation.

In this paper, we have demonstrated that Gene Regulatory Networks (GRNs) provide a proven architectural blueprint for distributed, stochastic information processing. By formalizing this analogy through Applied Category Theory, we have derived a comprehensive suite of design patterns:

Core Contributions

1. **Robustness via Topology:** The Coherent Feed-Forward Loop provides error suppression proportional to $(1 - \rho)$, where ρ is the correlation between component error modes. We make precise the conditions under which topological redundancy provides genuine safety benefits: diversity in model families yields $\rho \approx 0$; same-model verification yields $\rho \approx 1$ with minimal improvement. The topology is necessary but not sufficient; component diversity determines actual error suppression.

2. **Adaptive Immunity:** We formalize the Self/Non-Self distinction as a **Provenance Functor** $\mathcal{P} : \mathbf{Msg} \to \mathbf{Trust}$ with structurally-enforced labels. The Trust-Gated Lens provides injection resistance by ensuring that content-based attacks cannot elevate provenance. This extends the Prion metaphor into a full immunological framework with MHC-like tagging, negative selection during training, and regulatory suppression of conflicting sources.

3. **Epistemic Health:** The formalization of **Epiplexity** (Bayesian Surprise) as a metric for detecting "epistemic starvation." We provide an operational approximation using embedding similarity and conditional perplexity, with windowed detection to distinguish task completion from pathological loops. This connects agent dynamics to the Free Energy Principle: healthy agents minimize surprise through learning or effective action; stagnant agents do neither.

4. **Metabolic Intelligence:** The reframing of the Runtime from passive budget to active **Cognitive Control Policy**. Drawing on MIPS (Mitochondrial Information Processing System), we distinguish fast interventions (Apoptosis via mPTP-like triggers) from slow interventions (Retrograde Responses that reshape agent phenotype across sessions). The Runtime governs reasoning *quality* through the Metabolic-Epigenetic Coupling: low-budget states "methylate" expensive context, forcing efficient phenotypes.

5. **Multi-Cellular Organization:** The extension from single-agent to multi-agent systems using developmental biology. Agent phenotypes arise from differential context (epigenome) on shared weights (genome). Morphogen gradients (shared context variables) enable coordination without central control. Tissue boundaries enforce security isolation. This reframes "how many agents?" as "what is the developmental program?"

6. **Homeostasis:** Continuous self-repair through three modalities: Structural (Chaperone Loop with error-context feedback), Metabolic (Apoptosis + Regeneration with state summarization), and Cognitive (Autophagy via sleep/wake cycles). Most frameworks focus on Action; biology equally prioritizes Maintenance.

7. **Evolutionary Dynamics:** The Vermeij Trend predicts that agentic architectures face three selective pressures: adversarial robustness (Red Queen dynamics with attackers), task complexity (environmental pressure toward "aerobic" multi-step reasoning), and resource efficiency (metabolic selection toward the Pareto frontier). Architectures lacking proper immune defense and metabolic regulation will be outcompeted.

8. **Wire-Level Optics:** Prism optics enable conditional type-based routing (receptor specificity), Traversal optics enable batch processing (polymerase processivity), and DenatureFilters provide anti-injection sanitization—all composable on the same wire.

9. **Morphogen Diffusion:** A discrete-time dynamical system on the agent graph produces spatially varying concentration profiles, enabling position-dependent coordination without central control or global state sharing.

10. **Composable Coalgebras:** The coalgebraic formalism is made explicit with parallel and sequential composition, full transition traces, and bisimulation-based equivalence checking, enabling formal verification that two agent implementations are observationally interchangeable.

11. **Epistemic Topology:** Kripke-style knowledge operators (K_i, E_G, C_G, D_G) derived from wiring diagram structure yield four predictive theorems for multi-agent scaling—error amplification, sequential penalty, parallel acceleration, tool density—validated against empirical results from large-scale architecture evaluations [13].

12. **Capability-Gated Tool Acquisition:** The Plasmid Registry implements Horizontal Gene Transfer with capability gating, preventing privilege escalation when agents dynamically acquire tools.

13. **Diagram Optimization via Categorical Rewriting:** Cost annotations enrich the wiring category over the resource monoid $(\mathcal{R}, +, 0)$, making resource consumption a first-class compositional concept. Rewriting passes—dead wire elimination, parallel grouping, cost-order scheduling—are endofunctors that preserve input-output bisimulation while improving resource utilization. The ResourceAwareExecutor gates module execution on MetabolicState, connecting diagram optimization to the metabolic intelligence framework: under ATP depletion, non-essential pathway branches are downregulated, mirroring cellular starvation responses.

The Mathematical Foundation

The isomorphism rests on the category **Poly** of polynomial functors, where biological and software components at every scale—genes and capabilities, cells and agents, tissues and multi-agent systems—are modeled as interfaces (O, I) consuming observations and producing actions. The Operad of Wiring Diagrams provides the grammar for composition, with type-checking at the topological level preventing classes of runtime errors. The Metabolic Coalgebra enriches this with resource constraints, providing a decidable termination criterion when costs are strictly decreasing and regeneration is excluded (or separately bounded). The Provenance Functor layers trust semantics onto message flow, providing formal injection resistance.

Implications

The Operon framework is not merely a safety feature; it is the necessary evolutionary adaptation—the "vascularization" of software—that enables high-power cognition to function without thermodynamic collapse. The control structures that emerged over billions of years of evolution address

the same fundamental challenges of distributed, stochastic information processing that agentic architectures face today.

The isomorphism is not metaphorical; it is mathematical. And it is actionable: each biological motif maps to implementable code patterns, as demonstrated by the reference implementation. We anticipate that future agent frameworks will be evaluated not just on capability benchmarks, but on their metabolic efficiency, immune resistance, and homeostatic robustness—the same criteria that determine organismal fitness in biological evolution.

The epistemic topology formalization provides predictive power that has been validated against empirical scaling results [13], demonstrating that the framework captures real structure rather than post-hoc rationalization. The connection between morphogen-driven topology switching and epistemic optimization suggests a path toward formally verified adaptive multi-agent systems, where topological transitions are not merely heuristic but provably optimal under specified epistemic constraints.

Future Work

Several directions remain open:

- **Epiplexity Validation:** Empirical calibration of the α mixing parameter and threshold δ across diverse task types and LLM families. The current implementation uses a mock embedding provider; integration with production embedding APIs remains future work.
- **Correlation Estimation:** Lightweight methods for estimating ρ between agent error modes without exhaustive pairwise testing.
- **Distributed Diffusion:** The morphogen diffusion field currently operates in a single process. Extending to distributed multi-process deployments with eventual consistency remains open.
- **Adversarial Robustness:** Red-team evaluation of immune evasion vectors, including novel injection syntax, tool poisoning, and behavioral mimicry attacks. Development of continuous adaptation mechanisms (signature updates, thymic retraining, threat intelligence sharing).
- **Production Benchmarks:** Validation on real LLM outputs at scale, measuring both security efficacy and performance overhead of the defense layers.
- **Bisimulation at Scale:** The current bisimulation check operates over finite input sequences. Extending to infinite-trace coinductive bisimulation for continuous agent systems is an open challenge.
- **Learned Rewriting Rules:** The current optimization passes are hand-crafted. Learning rewriting rules from execution traces—identifying recurring subdiagram patterns and their optimized replacements—would connect diagram optimization to program synthesis and equality saturation techniques.

References

[1] Vincent Abbott. Neural circuit diagrams: Robust diagrams for the communication, implementation, and analysis of deep learning architectures. *arXiv preprint arXiv:2402.05424*, 2023. Foundation for the diagrammatic scheme used in FlashAttention on a Napkin.

[2] Vincent Abbott and Gioele Zardini. Flashattention on a napkin: A diagrammatic approach to deep learning io-awareness, 2025. Derives FlashAttention via Neural Circuit Diagrams with IO-aware performance model.

[3] John F Allen. Energy transduction and the mind: mitochondria in brain function. *The Biochemist*, 44(4):8–13, 2022.

[4] Uri Alon. Network motifs: theory and experimental approaches. *Nature Reviews Genetics*, 8(6):450–461, 2007.

[5] B.J. Aubrey, A. Strasser, and G.L. Kelly. How does p53 induce apoptosis and how does this relate to p53-mediated tumour suppression? *Cell Death & Differentiation*, 25:104–113, 2018.

[6] Michele Boreale. Coalgebras for bisimulation of weighted automata. *Logical Methods in Computer Science*, 19, 2023.

[7] Navdeep S Chandel. Mitochondria as signaling organelles. *BMC Biology*, 12:34, 2014. Revisited in 2024 work on metabolic-epigenetic coupling.

[8] Jesse Jiryu Davis. Reasoning about knowledge in TLA+, 2026. UCLA CS 201-A Seminar, MongoDB Distributed Systems Research Group.

[9] Edoardo Debenedetti, Giorgio Severi, Nicholas Carlini, Scott Coull, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *NeurIPS*, 2024.

[10] Ronald Fagin, Joseph Y Halpern, Yoram Moses, and Moshe Y Vardi. *Reasoning About Knowledge*. MIT Press, 1995.

[11] Brendan Fong and David I Spivak. *Seven sketches in compositionality: An invitation to applied category theory*. Cambridge University Press, 2019.

[12] Karl Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010.

[13] Google Research. Towards a science of scaling agent systems: When and why agent systems work. 2026. Available at: `https://research.google/blog/towards-a-science-of-scaling-agent-systems-when-and-why-agent-systems-work/`.

[14] Joseph Y Halpern and Yoram Moses. Knowledge and common knowledge in a distributed environment. *Journal of the ACM*, 37(3):549–587, 1990.

[15] Christopher P Kempes, David H Wolpert, Zachary Cohen, and Juan Pérez-Mercader. The thermodynamic efficiency of computations made in cells across the range of life. *Philosophical Transactions of the Royal Society A*, 375(2109), 2017.

[16] Michael Lynch and Georgi K Marinov. The bioenergetic costs of a gene. *Proceedings of the National Academy of Sciences*, 112(51):15690–15695, 2015.

[17] Humberto R Maturana and Francisco J Varela. *Autopoiesis and cognition: The realization of the living*. Reidel, 1980.

[18] Ron Milo, Shai Shen-Orr, Shalev Itzkovitz, Nadav Kashtan, Dmitri Chklovskii, and Uri Alon. Network motifs: simple building blocks of complex networks. *Science*, 298(5594):824–827, 2002.

[19] Georgi Nakov and Fredrik Nordvall Forsberg. Quantitative polynomial functors. In *Conference on Algebra and Coalgebra in Computer Science (CALCO)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2021.

[20] Nelson Niu and David I Spivak. Polynomial functors: A mathematical theory of interaction. *arXiv preprint arXiv:2312.00990*, 2023.

[21] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*, 2023.

[22] Martin Picard and Orian S Shirihai. The mitochondrial information processing system: A new model of mitochondrial-nuclear communication. *Trends in Neurosciences*, 41(4):206–208, 2018. Revisited and expanded in 2022-2024 work.

[23] Martin Picard and Orian S Shirihai. Mitochondria as multifaceted regulators of cell death. *Cell Metabolism*, 34(11):1607–1627, 2022.

[24] David I Spivak. Learners' languages. *Compositionality*, 3(4), 2021.

[25] Dmitry Vagner, David I Spivak, and Eugene Lerman. Algebras of open dynamical systems on the operad of wiring diagrams. *Theory and Applications of Categories*, 30(51):1793–1822, 2015.

[26] Geerat J Vermeij. *The Evolution of Power: A New Understanding of the History of Life*. Princeton University Press, 2023.

[27] Conrad H Waddington. *The Strategy of the Genes*. Allen & Unwin, 1957. Introduced the concept of the "epigenetic landscape".

[28] Jason Wei, Xuezhi Yi, Maarten Zhang, Yiming Liu, Xinyu Li, Xing Wang, Yujia Zhou, Yiming Li, Yuyang Wang, Zhi Li, et al. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022. Available at: `https://arxiv.org/abs/2201.11903`.